

Unidad 8

El perceptrón de Rosenblatt. Funciones de activación. Arquitectura de una red feedforward: Capas de entrada, capas ocultas (*hidden layers*) y capas de salida. El proceso de aprendizaje: Forward Propagation: Flujo de tensores a través de la red.

Función de Pérdida (Loss Function): Cross-entropy para clasificación y MSE para regresión.

Backpropagation: El algoritmo de la cadena (derivadas) para la actualización de pesos.

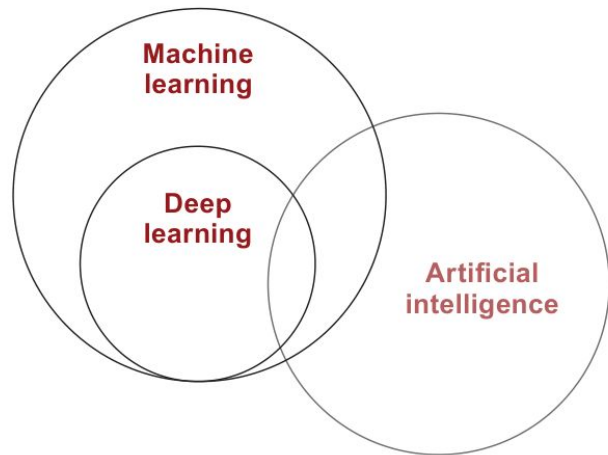
Generalidades y un poco de historia

Aprendizaje profundo

El aprendizaje profundo es el subconjunto del aprendizaje automático que se ocupa de modelos basados en redes neuronales multicapa.

Es profundo precisamente porque sus modelos aprenden múltiples capas de transformaciones. Si bien esto puede sonar limitado, el aprendizaje profundo ha dado lugar a una asombrosa variedad de modelos, técnicas, formulaciones de problemas y aplicaciones.

Podría decirse que todo aprendizaje automático tiene múltiples capas de computación, la primera de las cuales consiste en pasos de procesamiento de características. Lo que diferencia al aprendizaje profundo es que las operaciones aprendidas en cada una de las múltiples capas de representación se aprenden conjuntamente a partir de los datos.

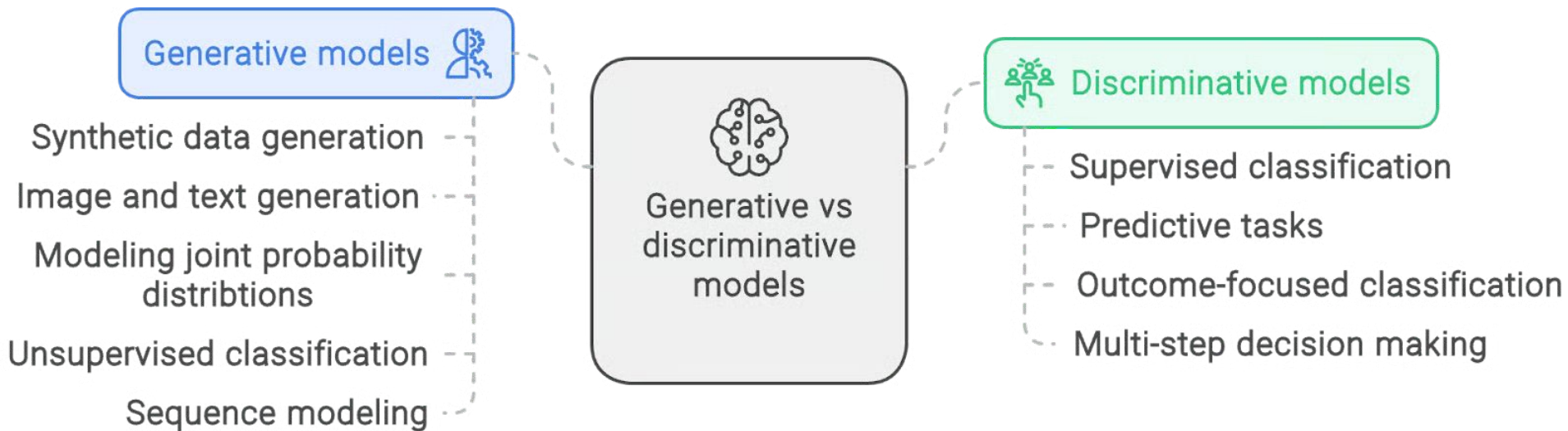


El **Teorema de Aproximación Universal (UAT)** es un principio matemático que establece que una red neuronal tiene la capacidad teórica de resolver cualquier problema computable o aproximar cualquier función continua a cualquier nivel de precisión deseado.

A nivel conceptual, como lo demuestra la versión del teorema propuesta por Cybenko, la idea básica es que cualquier función compleja o con muchas curvas puede aproximarse matemáticamente uniéndola como un conjunto de líneas rectas o como una suma de múltiples rectángulos (o "torres"). Para que la red se acerque más a la función real que intenta imitar, simplemente necesita usar líneas más cortas, lo que le permite modelar un conjunto sumamente rico de funciones utilizando un número finito de neuronas.

- **Problema del tamaño:** es posible representar cualquier cosa utilizando una red neuronal que tenga una sola capa oculta (y una sola función no lineal). La trampa de esto es que no pone un límite a la cantidad de neuronas necesarias en esa capa; mientras más complicada sea la función objetivo, más neuronas se requerirán, lo que puede resultar en redes inmensas e imposibles de manejar en la vida real
- **Teoría vs Aprendizaje:** Existe una diferencia fundamental entre lo que una red puede aprender y representar *en principio* según las matemáticas, y lo que realmente le resulta fácil aprender en la práctica utilizando algoritmos de optimización y datos del mundo real.

Modelos discriminativos y modelos generativos



Época: Número de veces que el algoritmo se ejecuta en todo el conjunto de datos de entrenamiento.

Muestra: Una sola fila de un conjunto de datos.

Lote: Indica el número de muestras que se deben tomar para actualizar los parámetros del modelo.

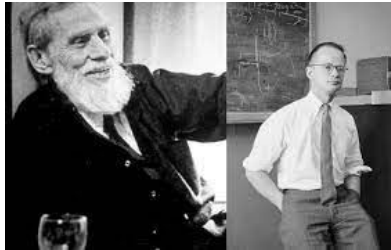
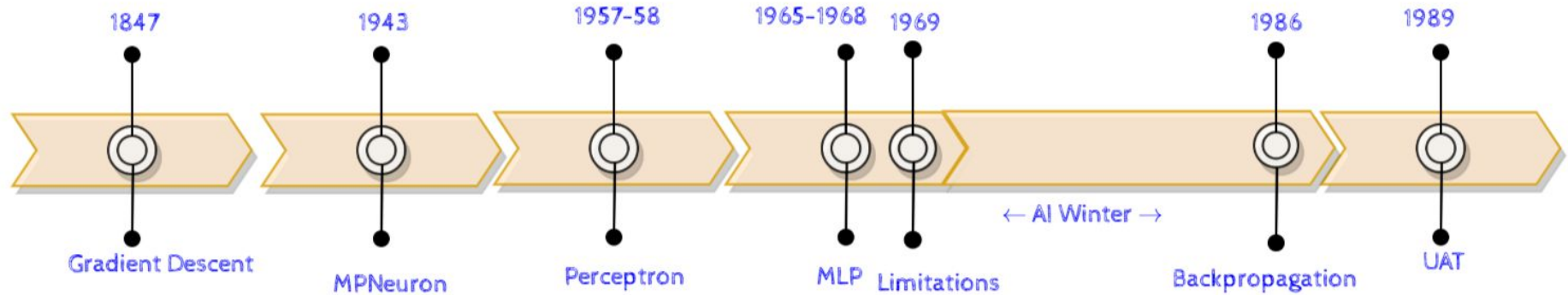
Tasa de aprendizaje: Es un parámetro que proporciona al modelo una escala de cuánto deben actualizarse los pesos del modelo.

Función de coste/Función de pérdida: Se utiliza una función de coste para calcular el coste, que es la diferencia entre el valor predicho y el valor real.

Pesos/Sesgo: Los parámetros aprendibles en un modelo que controlan la señal entre dos neuronas.

Un poco de historia

Una de varias líneas de tiempo



Warren McCulloch &
Walter Pitts



Frank Rosenblatt

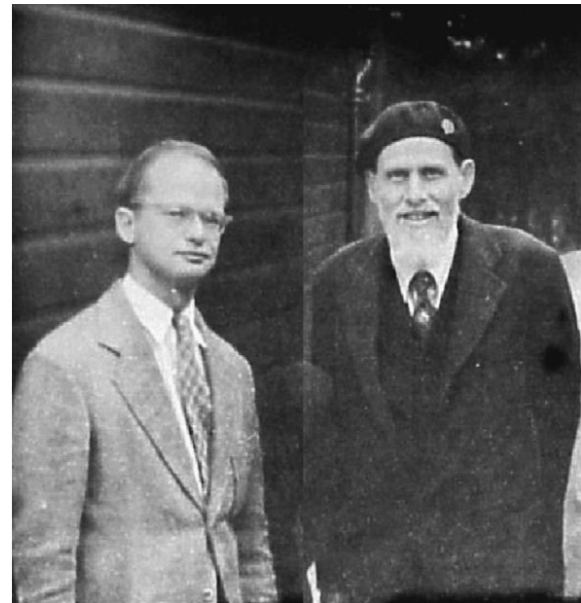


Marvin Minsky and
Seymour Papert

Un poco de historia...

En 1943, el neurofisiólogo y cibernético estadounidense **Warren McCulloch**, de la Universidad de Illinois en Chicago, y el lógico autodidacta y psicólogo cognitivo **Walter Pitts** publicaron «Un cálculo lógico de las ideas inmanentes en la actividad nerviosa», donde describen la «neurona de McCulloch-Pitts», el primer modelo matemático de una red neuronal.

Basándose en las ideas de «Sobre los números computables» de Alan Turing, el artículo de McCulloch y Pitts proporcionó una forma de describir las funciones cerebrales en términos abstractos y demostró que los elementos simples conectados en una red neuronal pueden tener una enorme capacidad de cálculo. El artículo recibió poca atención hasta que sus ideas fueron aplicadas por John von Neumann, Norbert Wiener y otros.



Inicios

Un poco de historia...

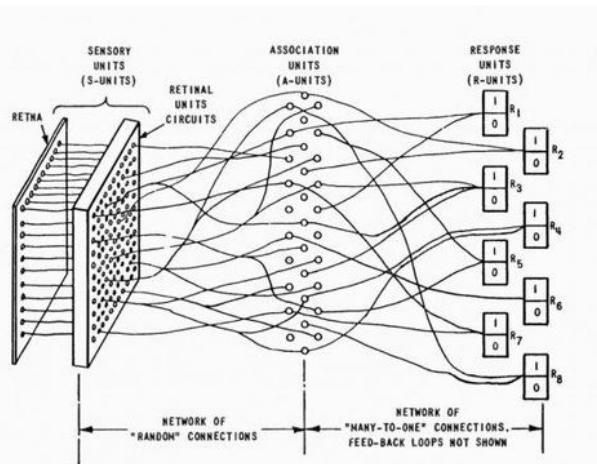
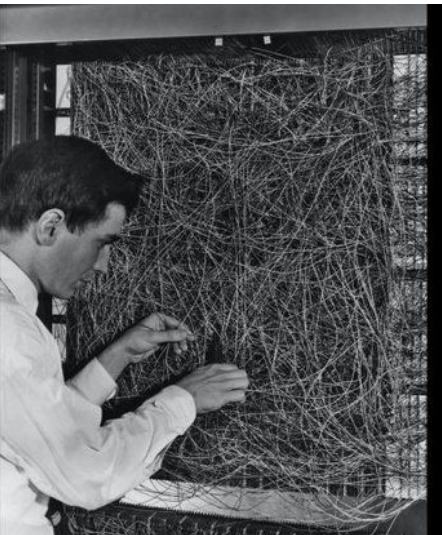


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON

El **perceptrón de Rosenblatt** no es solo un precursor histórico, sino el modelo fundacional que estableció los componentes esenciales de cualquier red neuronal moderna: entradas, pesos, una función de agregación y una función de activación. En su concepción original para el reconocimiento de patrones visuales, Rosenblatt diseñó el Mark I Perceptron, un sistema de hardware que utilizaba una "retina" de 400 fotocélulas conectadas a unidades de asociación que finalmente convergen en una capa de respuesta.

Perceptron de Rosenblatt

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

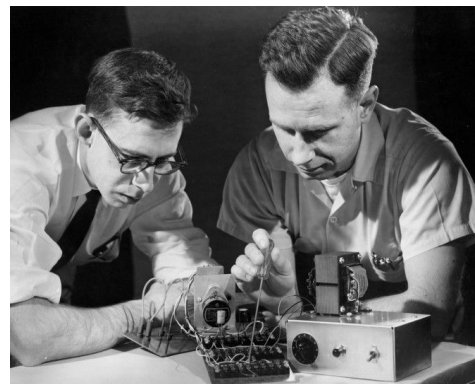
F. ROSENBLATT

Cornell Aeronautical Laboratory



Más info:

<https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon>

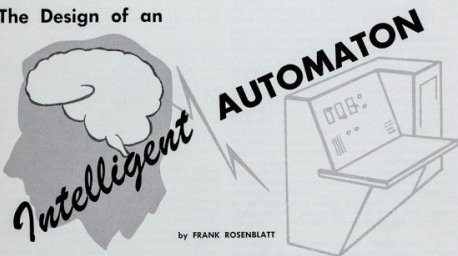


Vol. VI, No. 2, Summer 1958

research trends

CORNELL AERONAUTICAL LABORATORY, INC., BUFFALO 21, NEW YORK

The Design of an



by FRANK ROSENBLATT

Introducing the perceptron — A machine which senses, recognizes, remembers, and responds like the human mind.

STORIES about the creation of machines having human qualities have long been a fascinating province in the realm of science fiction. Yet we are now about to witness the birth of such a machine — a machine capable of perceiving, recognizing, and identifying its surroundings without any human training or control.

Development of that machine has stemmed from a search for an understanding of the physical mechanisms which underlie human experience and intelligence. The question of the nature of these processes is at least as ancient as any other question in western science and philosophy, and, indeed, ranks as one of the greatest scientific challenges of our time.

Our understanding of this problem has gone perhaps as far as had the development of physics before Newton. We have some excellent descriptions of the phenomena to be explained, a number of interesting hypotheses, and a little detailed knowledge about events in the nervous system. But we lack agreement on any integrated set of principles by which the functioning of the nervous system can be understood.

We believe now that this ancient problem is about to yield to our theoretical investigation for three reasons:

First, in recent years our knowledge of the functioning of individual cells in the central nervous system has vastly increased.

Second, large numbers of engineers and mathematicians are, for the first time, undertaking serious study of the mathematical basis for thinking, perception, and the handling of information by the central nervous system, thus providing the hope that these problems may be within our intellectual grasp.

Third, recent developments in probability theory and in the mathematics of random processes provide new tools for the study of events in the nervous system, where only the gross statistical organization is known and the precise cell-by-cell "wiring diagram" may never be obtained.

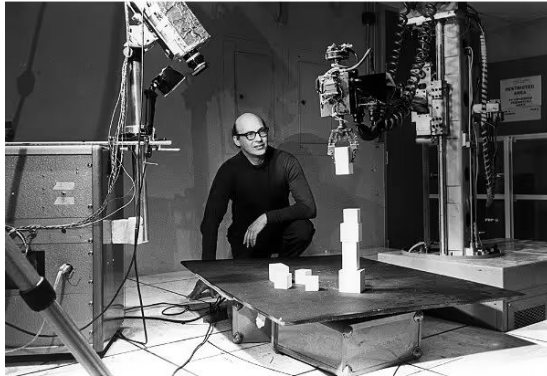
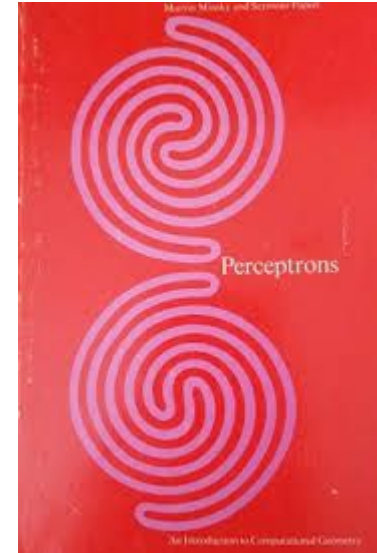
Receives Navy Support

In July, 1957, Project PARA (Perceiving and Recognizing Automaton), an internal research program which had been in progress for over a year at Cornell Aeronautical Laboratory, received the support of the Office of Naval Research. The program had been concerned primarily with the application of probability theory to

Críticas al perceptrón de Rosenblatt



Más información:
<https://yuxi.ml/essays/perceptron-controversy>

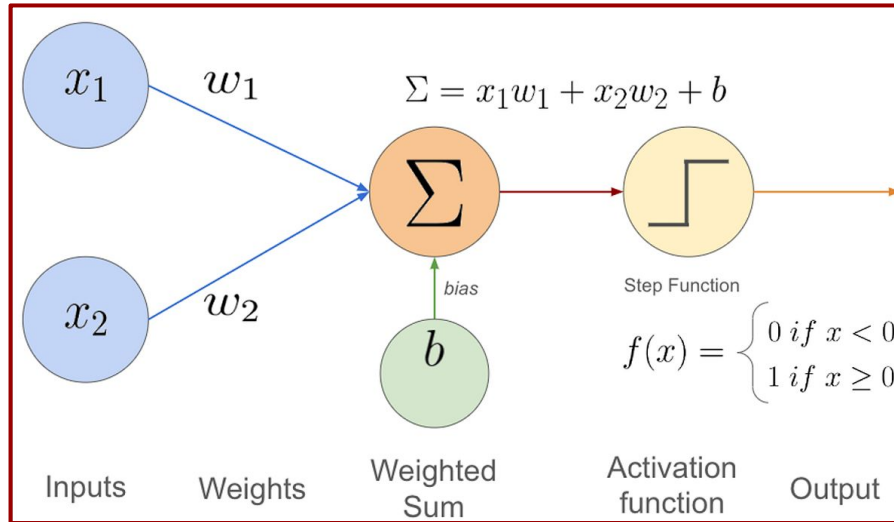


Diferencia entre
perceptrón de McCulloch
& Pitt y perceptron de
Rosenblatt

<https://goo.su/bsxHpFm>

Arquitecturas en Deep Learning

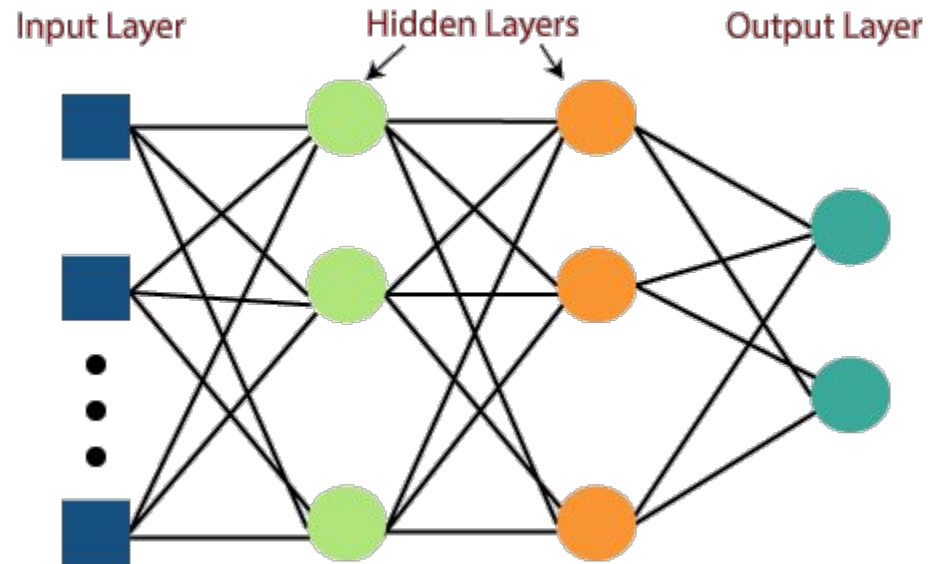
Tipos de redes neuronales



El **perceptrón simple** (o de una sola neurona) puede definirse como el bloque de construcción más básico y fundamental de las redes neuronales artificiales. Fue inventado por el psicólogo Frank Rosenblatt en 1958 y se considera el ancestro conceptual de todos los modelos modernos de *deep learning*.

Un perceptrón de una neurona es un **clasificador lineal binario**. Su propósito es tomar datos y decidir a cuál de dos categorías posibles pertenecen (por ejemplo, "sí" o "no", "1" o "0") dibujando una línea recta (o un hiperplano en múltiples dimensiones) que divida los datos como una frontera de decisión.

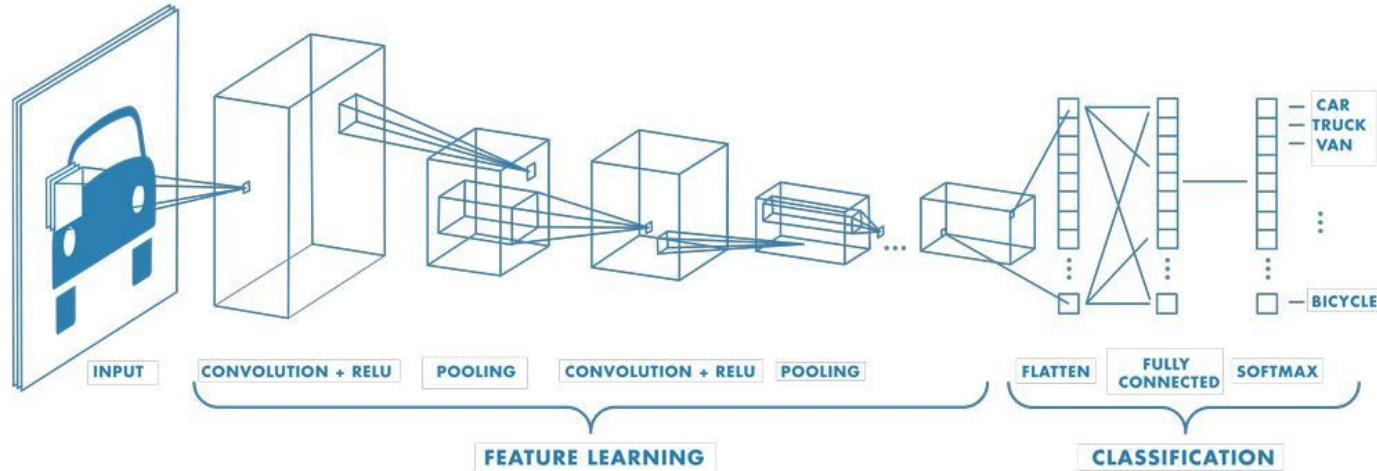
Tipos de redes neuronales



El Perceptrón Multicapa (MLP, por sus siglas en inglés) puede definirse como el modelo de red neuronal artificial más fundamental y clásico, constituyendo la base teórica y práctica sobre la cual se construye el aprendizaje profundo. También se le conoce frecuentemente en la literatura como red neuronal *feedforward* (prealimentada) o red densamente conectada.

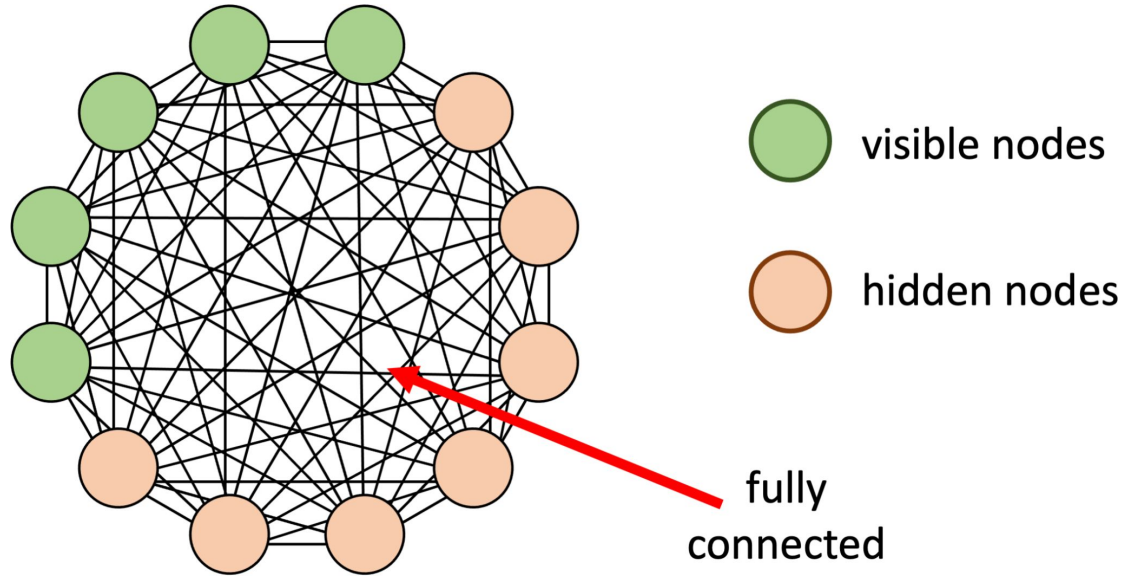
Debido a esta combinación de suma de pesos y activación no lineal en múltiples neuronas, el MLP cumple con el "Teorema de Aproximación Universal". Esto garantiza teóricamente que un MLP tiene la expresividad necesaria para aproximar cualquier función continua del mundo real con el grado de precisión deseado, siempre y cuando cuente con suficientes neuronas.

Tipos de redes neuronales



Las **Redes Neuronales Convolucionales** (conocidas como CNN o ConvNets, por sus siglas en inglés) son un tipo especializado de red neuronal artificial diseñado específicamente para procesar datos que tienen una topología o estructura de cuadrícula conocida, como datos de series temporales (cuadrícula unidimensional) o, más comúnmente, imágenes (cuadrícula bidimensional de píxeles).

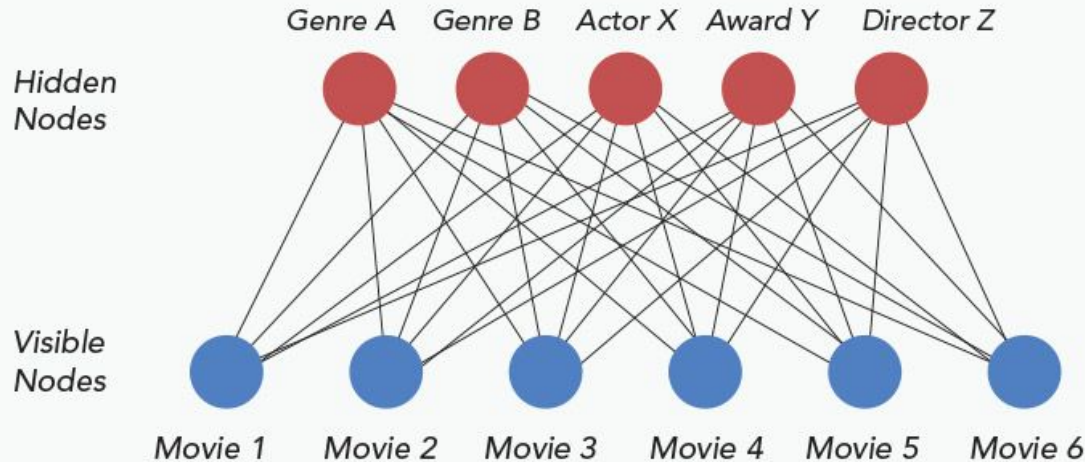
Tipos de redes neuronales



Las **Máquinas de Boltzmann (Boltzmann Machines)** son redes neuronales de conexiones simétricas que, a diferencia de otras arquitecturas clásicas, incluyen unidades (o neuronas) ocultas. Se consideran el modelo paradigmático para el aprendizaje estadístico y son capaces de aprender de manera eficiente un modelo generativo a partir de datos observados.

En su forma más simple, los nodos de una Máquina de Boltzmann se dividen en dos clases o capas: las **unidades visibles** y las **unidades ocultas**. Las unidades visibles se encargan de codificar la entrada y la salida de la máquina, mientras que las unidades ocultas se encargan de construir una "imagen interna" o representación de los datos

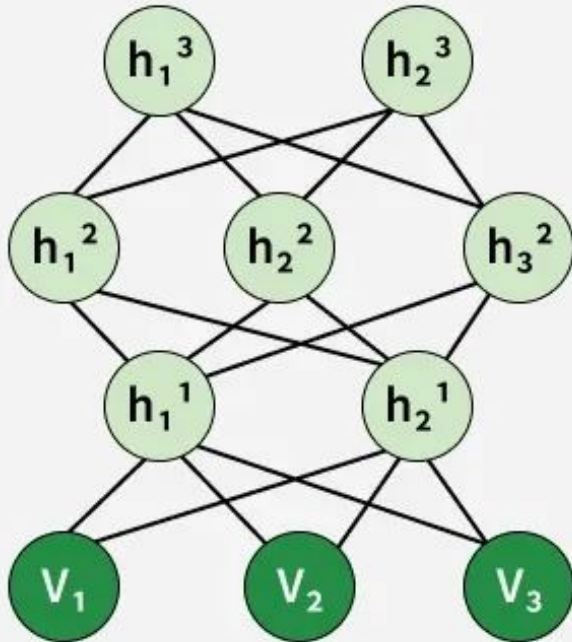
Tipos de redes neuronales



Máquina de Boltzmann Restringida (RBM, por sus siglas en inglés) es una variante especializada de la Máquina de Boltzmann clásica en la que se ha restringido la conectividad para facilitar y acelerar enormemente los procesos de inferencia y aprendizaje.

Una RBM consta de una sola capa de unidades visibles y una sola capa de unidades ocultas. La restricción fundamental de esta arquitectura es que **no existen conexiones entre las unidades de una misma capa** (las unidades ocultas no se conectan con otras unidades ocultas, y las visibles no se conectan con otras visibles). Las RBM se entrenan típicamente utilizando un atajo algorítmico muy eficiente llamado **Divergencia Contrastiva (Contrastive Divergence)**.

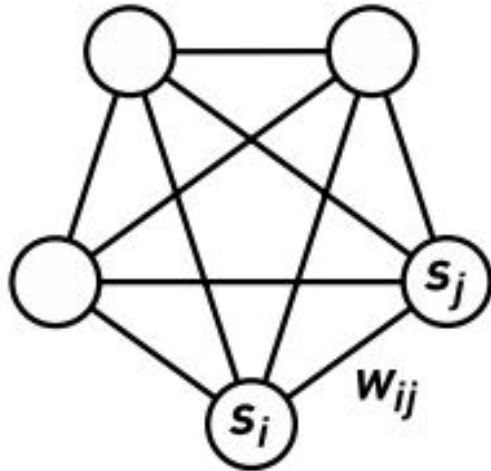
Deep Belief Network



Una **Red de Creencia Profunda** (Deep Belief Network o DBN, por sus siglas en inglés) es un tipo de red neuronal probabilística y generativa multicapa que se utiliza en el aprendizaje profundo para descubrir automáticamente patrones complejos en grandes conjuntos de datos.

Fueron introducidas alrededor de 2006 y representaron un gran avance histórico en el campo, ya que popularizaron el concepto de pre-entrenar redes neuronales profundas utilizando aprendizaje no supervisado.

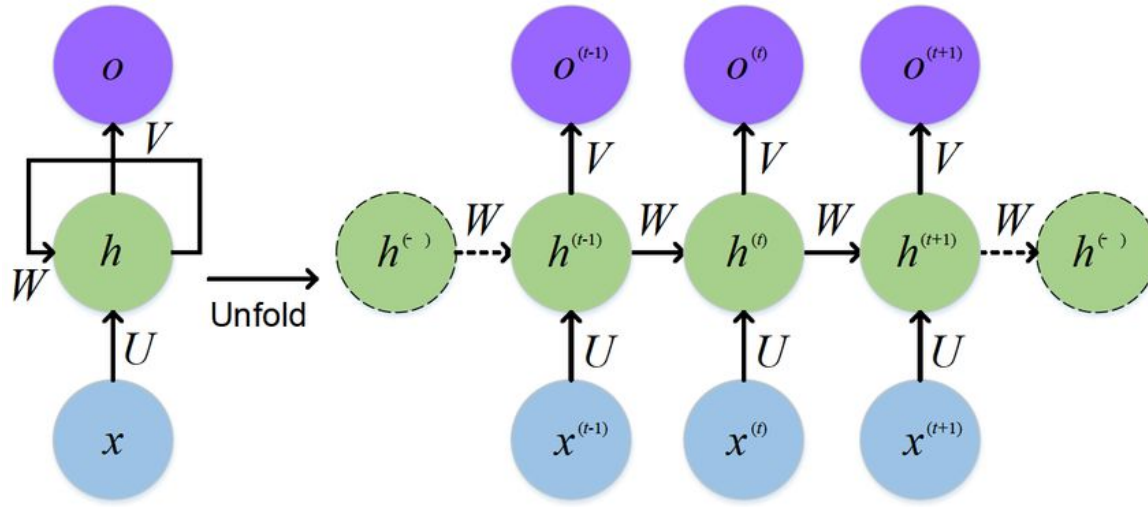
Hopfield network



Una **red de Hopfield** es un tipo de red neuronal artificial recurrente compuesta por unidades (neuronas) de umbral binario con conexiones simétricas entre ellas, y que, a diferencia de arquitecturas como las Máquinas de Boltzmann, carece de unidades ocultas.

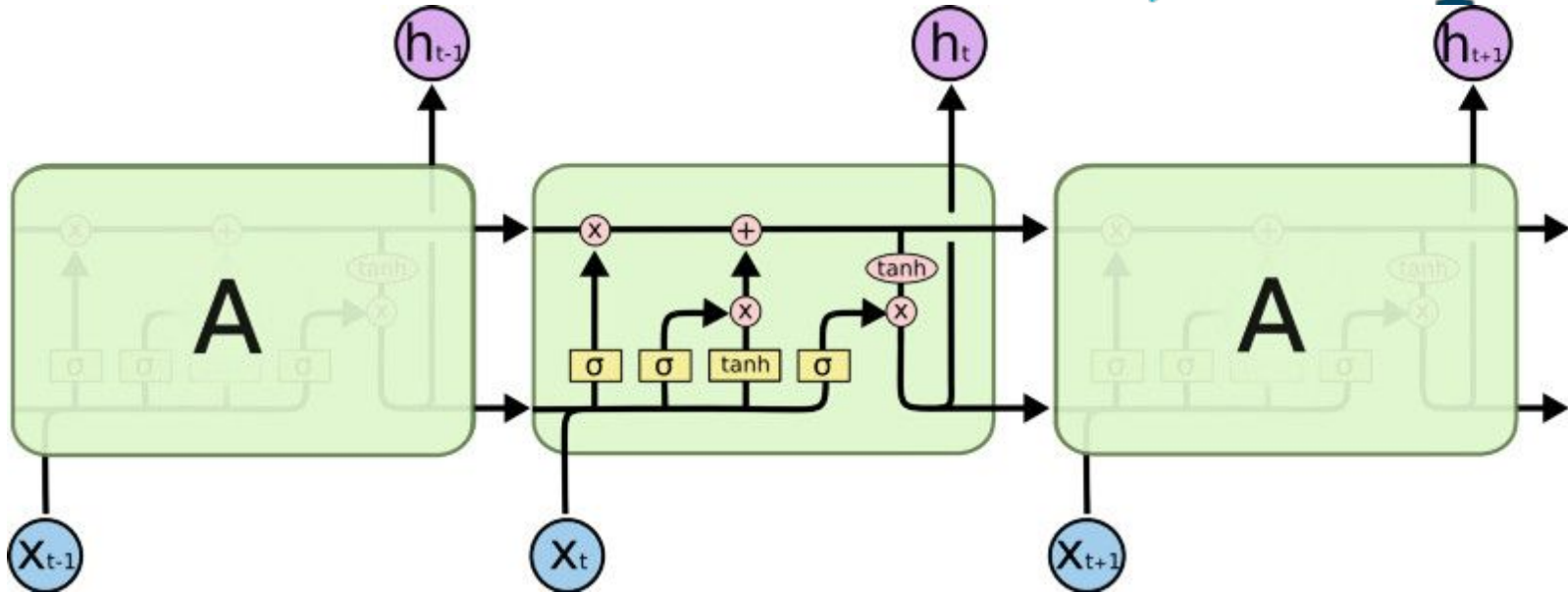
Su objetivo principal es la recuperación de la información. La red busca reconocer y reconstruir vectores de entrada (patrones o memorias) que ha aprendido previamente, pudiendo "limpiarlos" y completarlos de manera robusta incluso si los datos de entrada se le presentan en una versión ruidosa, corrupta o incompleta (funcionando como si reconstruyera un dinosaurio a partir de unos pocos huesos).

Tipos de redes neuronales



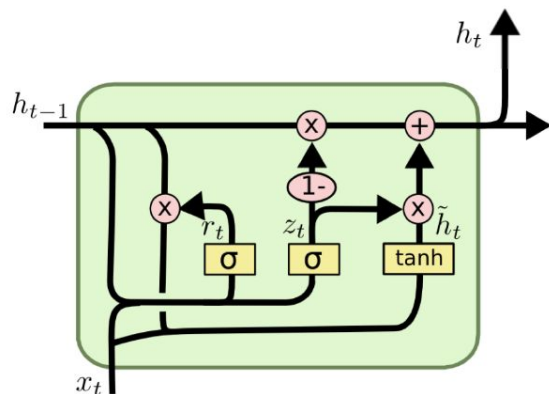
Las **Redes Neuronales Recurrentes (RNN, por sus siglas en inglés)** pueden definirse como una familia de redes neuronales diseñada específicamente para procesar datos secuenciales, como texto, audio o series temporales. A diferencia de otros tipos de redes, **tiene memoria**. Esa memoria en este tipo de redes tenía el inconveniente de perderse en un proceso que se llama **vanishing gradients/exploding gradients**. Eso se soluciona mediante el **mecanismo de atención**.

Tipos de redes neuronales



Las **LSTM (Long Short-Term Memory o Memoria a Corto y Largo Plazo)** son una arquitectura de redes neuronales recurrentes (RNN) introducida en 1997 por los investigadores Jürgen Schmidhuber y Sepp Hochreiter. Fueron diseñadas con el propósito específico de resolver los problemas de desvanecimiento y explosión de gradientes que limitan a las RNN tradicionales, permitiendo así que la red pueda retener y transmitir de forma confiable información importante a través de muchos pasos de tiempo hacia el futuro.

Tipos de redes neuronales



GRU
(Gated Recurrent Unit)

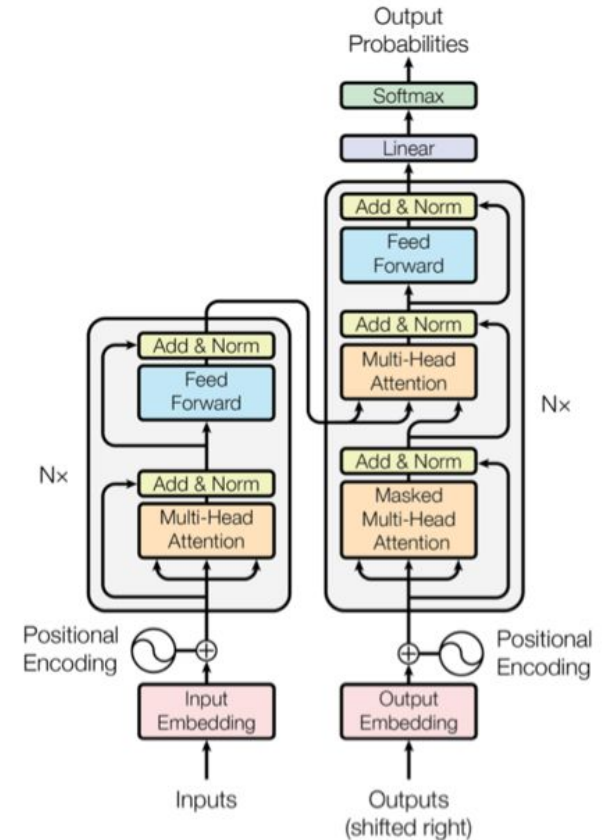
Las **Gated Recurrent Units (GRU)**, o Unidades Recurrentes con Compuertas, son un tipo de capa utilizada en las redes neuronales recurrentes (RNN) para el procesamiento de secuencias.

Se puede definir a una GRU como una **versión ligeramente más simple y optimizada de la arquitectura LSTM** (Long Short-Term Memory). Al igual que las LSTM, las unidades GRU fueron diseñadas específicamente para **resolver los problemas de desvanecimiento y explosión de gradientes** (y activaciones) que sufren las redes recurrentes tradicionales o "vanilla" cuando operan sobre secuencias largas.

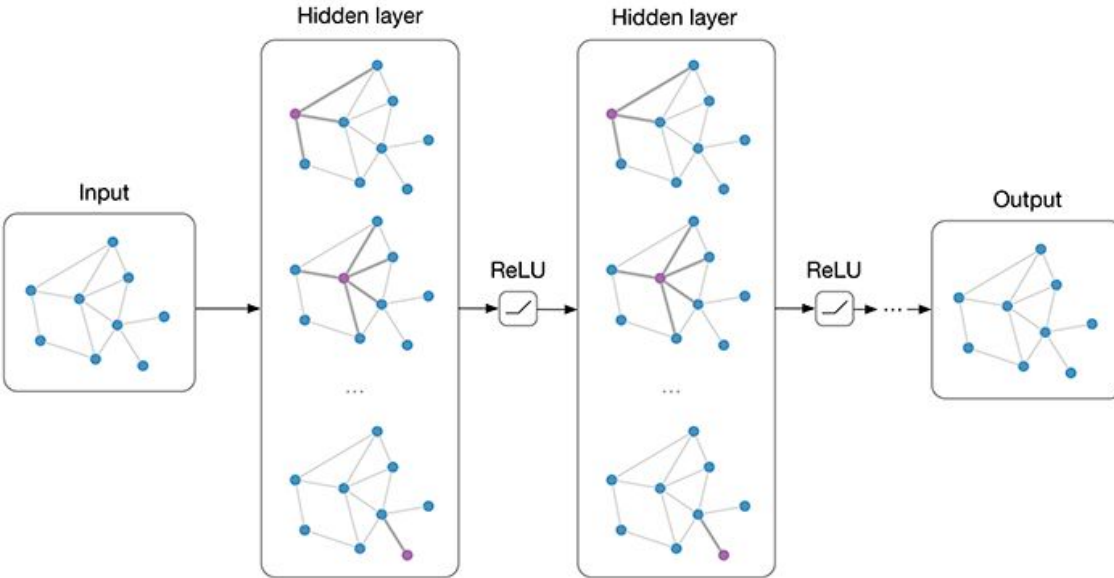
Gracias a este diseño, las GRU permiten que la red pueda aprender dependencias a largo plazo, reteniendo la información importante a lo largo de muchos pasos de tiempo en la secuencia, pero lográndolo de una manera más simplificada en comparación con una LSTM.

Tipos de redes neuronales

Los **Transformers** son una arquitectura de redes neuronales muy potente diseñada originalmente para resolver problemas de aprendizaje de secuencia a secuencia (sequence-to-sequence), como la traducción automática. Desde su introducción entre 2017 y 2018, han revolucionado el campo del procesamiento de lenguaje natural (NLP), reemplazando en gran medida a las redes neuronales recurrentes (RNN) como el estándar principal



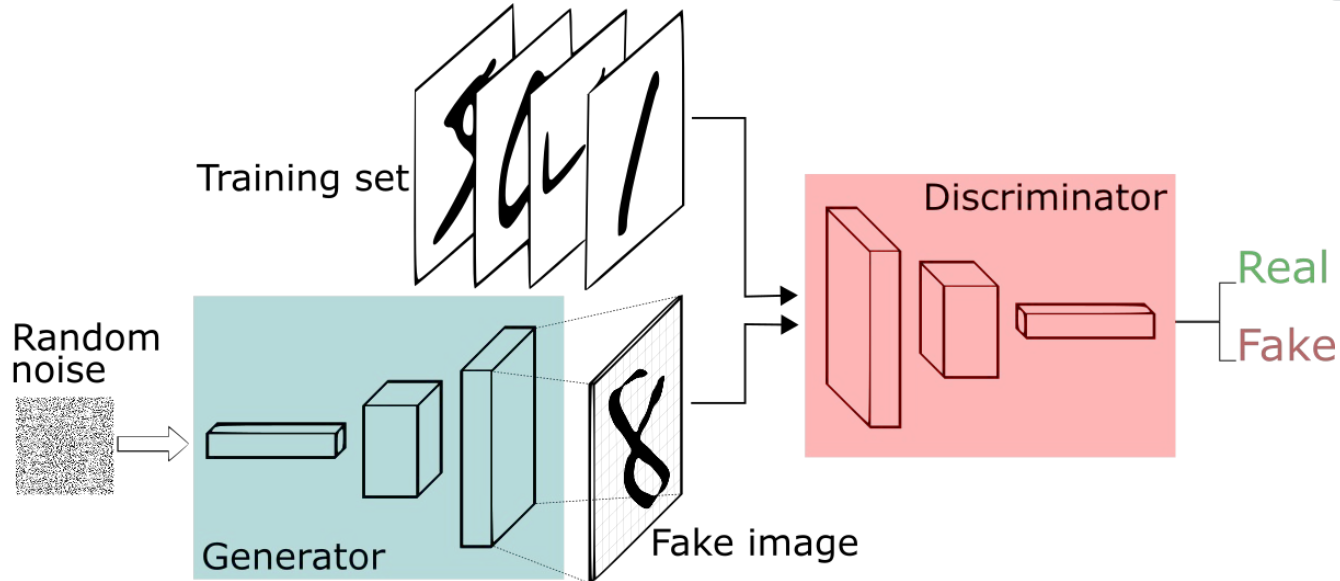
Tipos de redes neuronales



Las **redes neuronales basadas en grafos (GNN)** son redes neuronales diseñadas para operar sobre datos estructurados como **grafos** $G=(V,E)$, donde V es el conjunto de nodos y E el de aristas. A diferencia de las redes tradicionales que asumen datos en formato euclidiano (vectores, imágenes), las GNN aprenden representaciones de nodos, aristas o grafos completos respetando la **topología de las conexiones**.

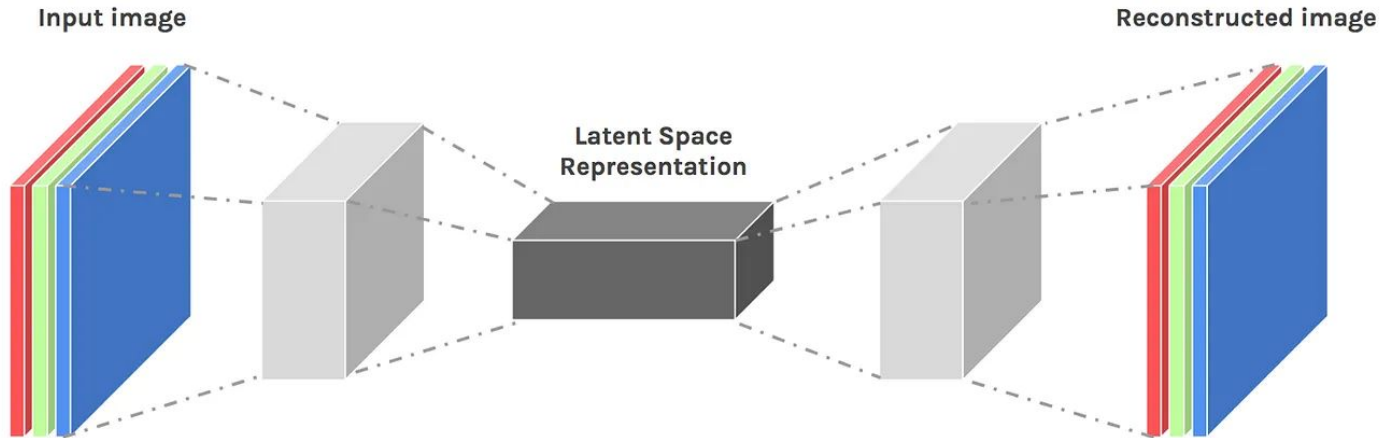
Aplicaciones típicas: detección de fraude, moléculas, redes sociales, grafos de conocimiento.

Tipos de redes neuronales



Las **Redes Generativas Adversariales (GANs)** son una arquitectura de aprendizaje profundo introducida en 2014 cuyo objetivo principal es la **creación de datos sintéticos** (como imágenes, audio o texto) que resulten estadísticamente indistinguibles de los datos reales.

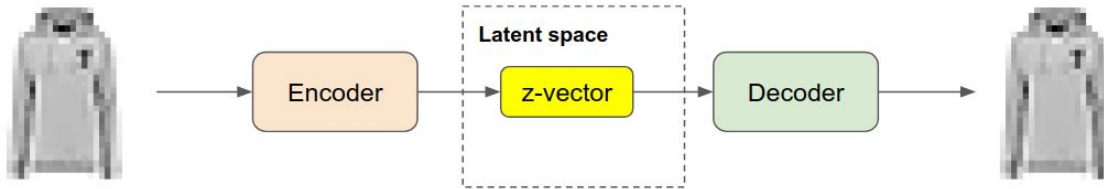
Tipos de redes neuronales



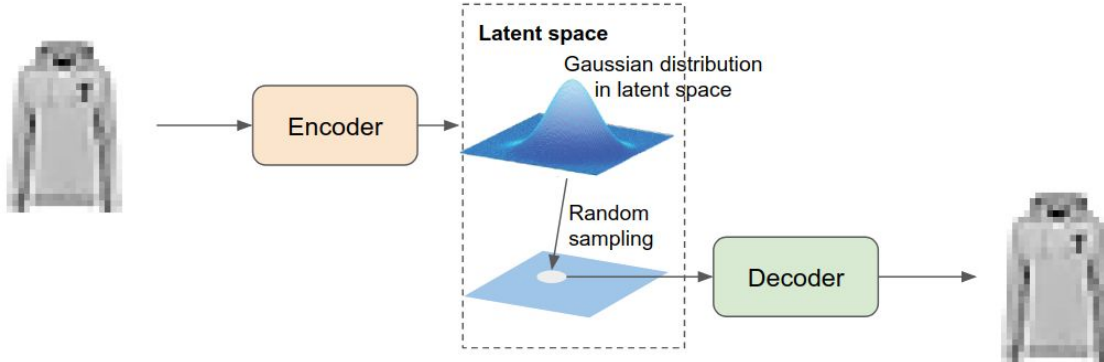
Un **Autoencoder** (o autoencodificador) es una arquitectura de red neuronal diseñada para aprender de forma no supervisada representaciones compactas y de baja dimensionalidad (conocidas como *embeddings* o códigos) a partir de los datos. Los Autoencoders son redes utilizadas principalmente en el aprendizaje no supervisado para extraer características útiles, comprimir datos o reconstruir señales ruidosas (por ejemplo, los *denoising autoencoders* o *noisy autoencoders*).

Tipos de redes neuronales

A. How a classical autoencoder works



B. How a variational autoencoder works

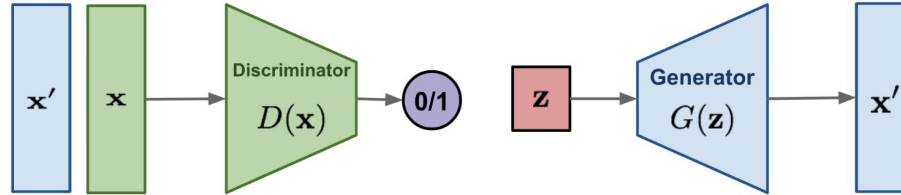


Un **Variational Autoencoder (VAE)** (o autoencodificador variacional) es un modelo generativo que representa una evolución basada en la probabilidad y la inferencia bayesiana del autoencoder clásico. Fue diseñado para resolver el problema de que los autoencoders clásicos no generan espacios latentes bien estructurados o continuos.

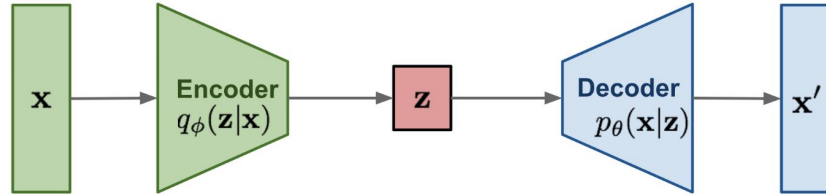
Gracias a la aleatoriedad y a la regularización, **los VAE garantizan que el espacio latente sea continuo, suave y altamente estructurado**. Esto significa que cualquier punto cercano en ese espacio latente se decodificará en una imagen válida y muy similar a las de su alrededor.

Tipos de redes neuronales

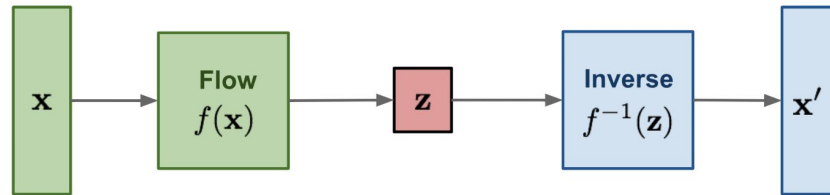
GAN: minimax the classification error loss.



VAE: maximize ELBO.



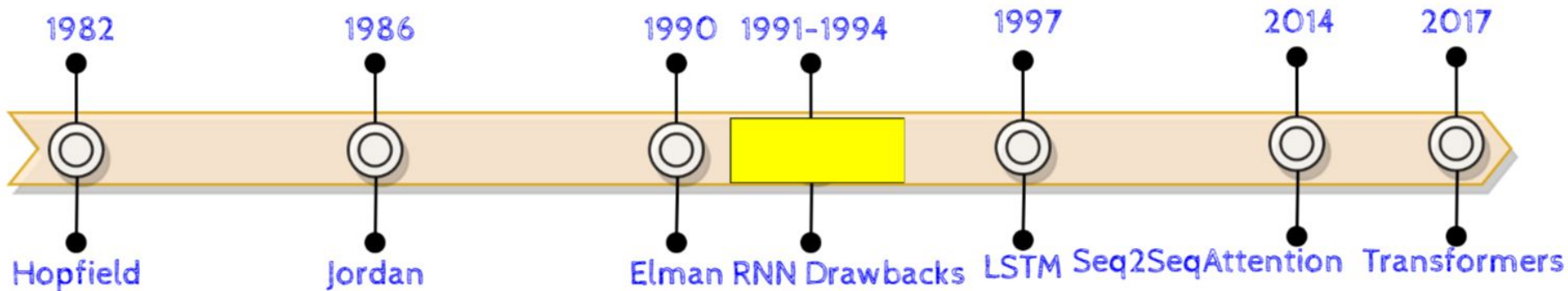
Flow-based generative models: minimize the negative log-likelihood

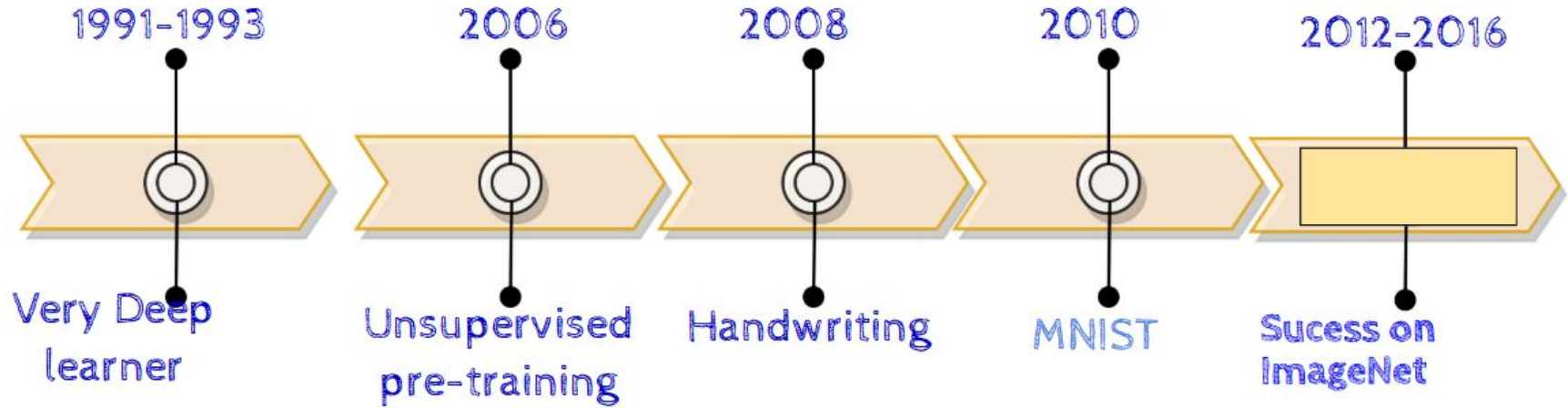


Los **Normalizing Flows** (Flujos Normalizadores) son una familia de modelos generativos que aprenden explícitamente la distribución de probabilidad real de los datos.

Funcionan transformando una distribución de probabilidad simple y conocida (como una distribución normal o gaussiana estándar) en una distribución mucho más compleja (la de los datos reales) mediante una **secuencia de transformaciones matemáticas que deben ser estrictamente diferenciables e invertibles**.

Redes basadas en secuencias





Perceptrón de Rosenblatt

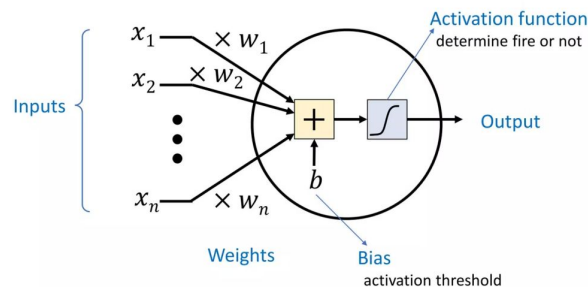
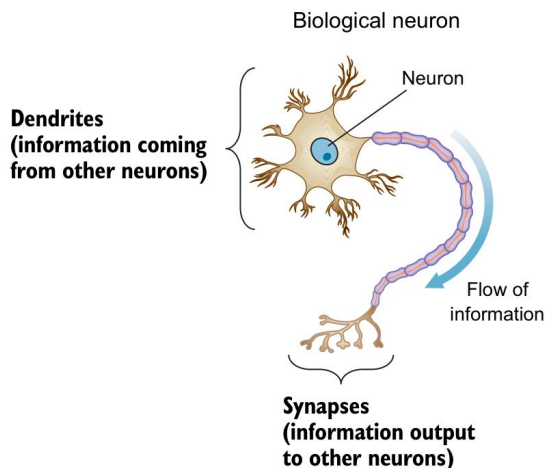
Red Neuronal de una sola neurona: Perceptron de Rosenblatt

Una **red neuronal** (o red neuronal artificial) es un modelo computacional inspirado en la estructura y el funcionamiento del sistema nervioso humano. Es la unidad fundamental de lo que hoy conocemos como Deep Learning.

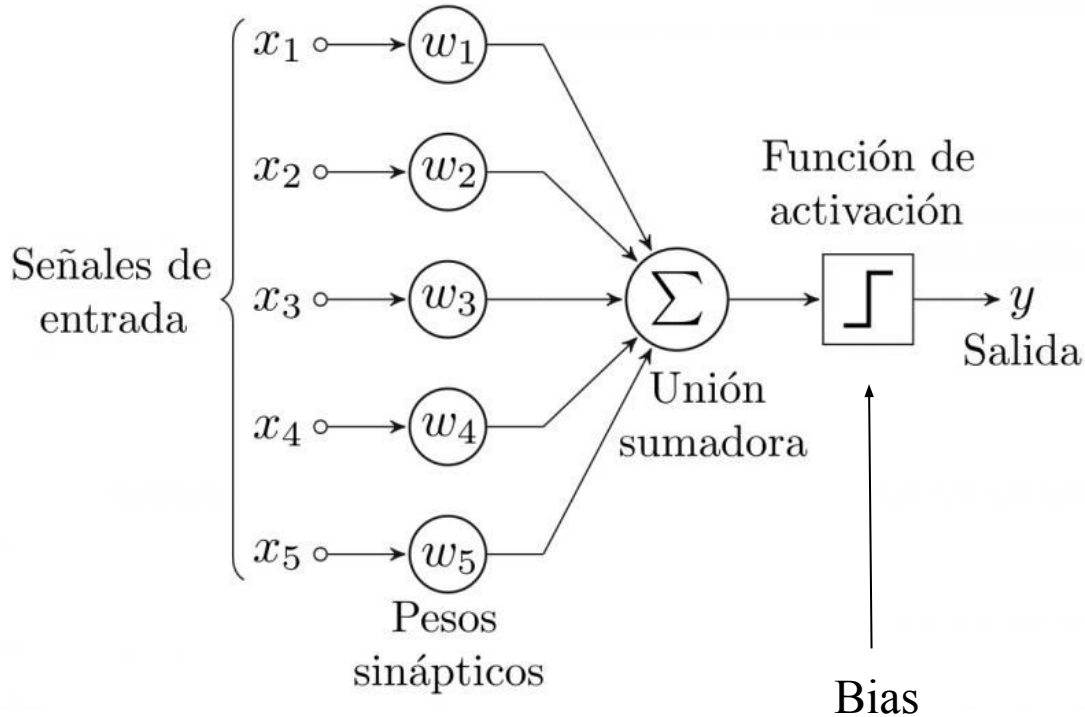
Es un algoritmo diseñado para **reconocer patrones**.

Componentes:

- **Capa de Entrada (Input Layer):** Es donde el modelo recibe los datos brutos (por ejemplo, los píxeles de una foto).
- **Capas Ocultas (Hidden Layers):** Son capas intermedias donde ocurre el procesamiento matemático. Aquí es donde la red extrae características abstractas de los datos.
- **Capa de Salida (Output Layer):** Es el resultado final, como una etiqueta que dice "Gato /Perro" o una predicción de precio.



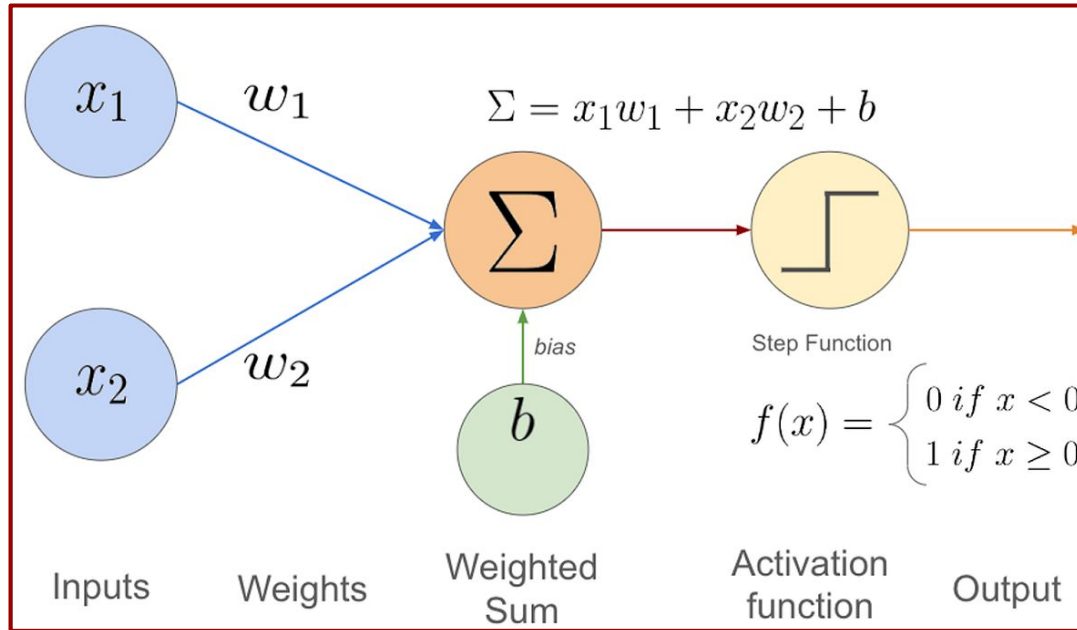
Perceptrón de Rosenblatt



Partes:

- Input o datos de entrada
- Bias
- Pesos o weights
- Función de agregación (weighted sum)
- Función de activación

Arquitectura de una red neuronal



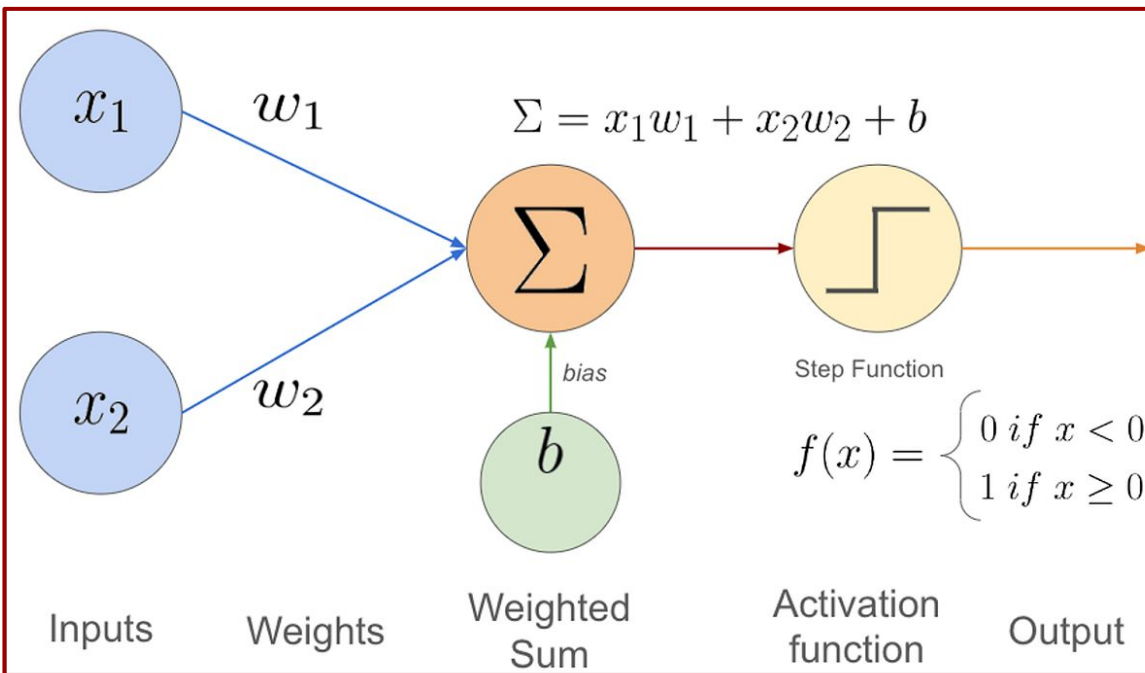
Los pesos (weights) son variables o parámetros numéricos asociados a cada una de las conexiones entre las neuronas.

Propósito: Determinar la influencia o importancia de cada entrada: El propósito de este valor es determinar exactamente qué tanta contribución o influencia tiene esa entrada específica sobre el resultado final de la neurona

Almacenar el conocimiento: Los pesos son, en esencia, la "memoria" de la red.

Son el centro del "Aprendizaje": En el Machine Learning, "aprender" significa literalmente encontrar el conjunto de valores correctos para estos pesos.

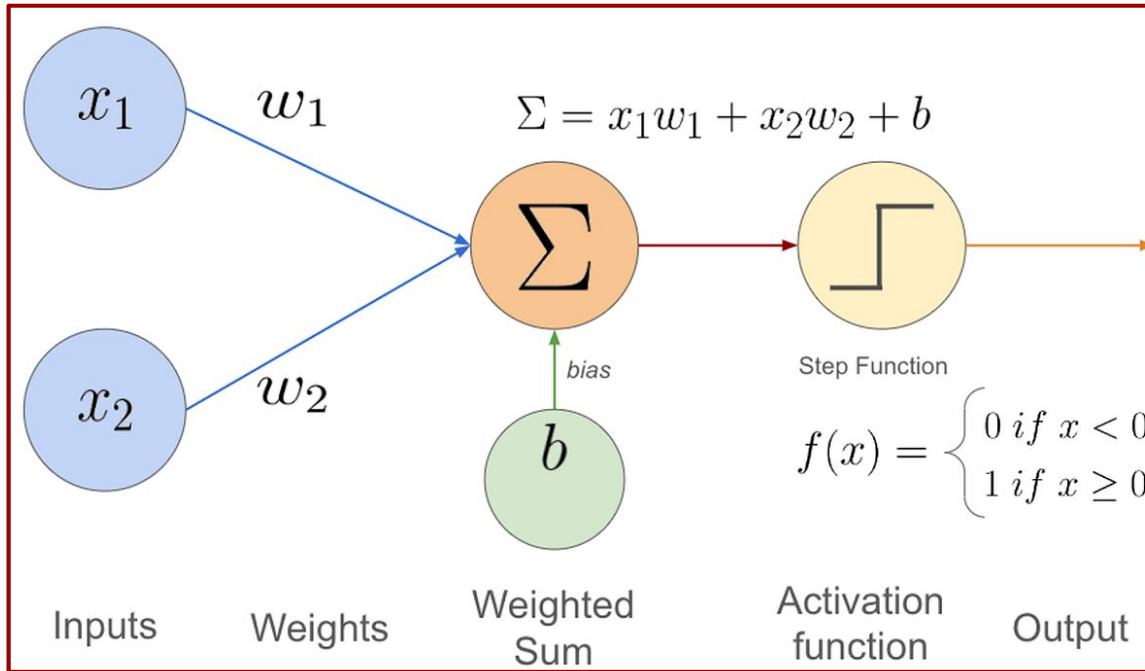
Arquitectura de una red neuronal



En un perceptrón (o neurona artificial), el **bias** (sesgo) es un parámetro matemático constante, comúnmente denotado como b , que se suma al producto de las entradas y sus respectivos pesos antes de que el resultado pase por la función de activación.

En los modelos clásicos, **el bias es exactamente equivalente a tener un "umbral de activación" negativo**. En lugar de decir que la suma ponderada debe superar un cierto umbral para que la neurona dispare, se suma el bias a la ecuación y la neurona simplemente evalúa si el total supera el valor de cero.

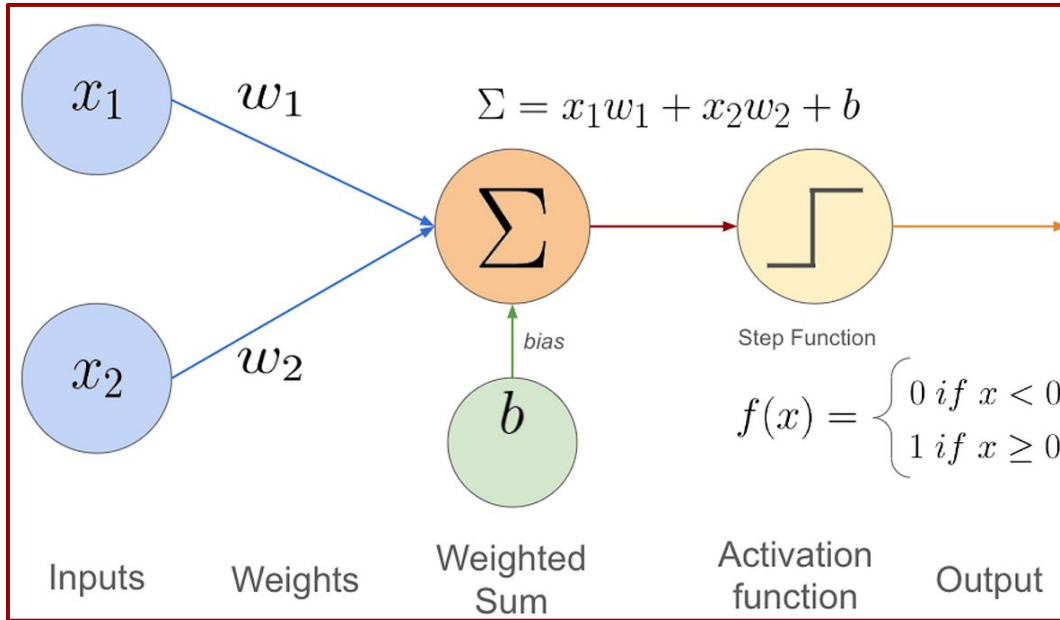
Arquitectura de una red neuronal



La función de agregación o suma ponderada de las entradas y la función de activación son los pasos que definen su salida.

Propósito de la suma ponderada: mide la relevancia combinada de todas las entradas. Las entradas con pesos más altos contribuyen más a este valor agregado, lo que refleja su importancia en la toma de decisión de la neurona.

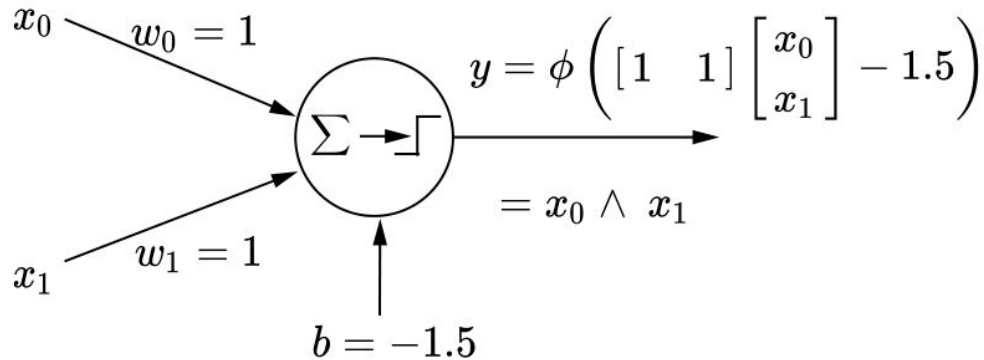
Arquitectura de una red neuronal



La función de activación en el Perceptrón de Rosenblatt es el componente esencial que toma la suma ponderada de todas las entradas (multiplicadas por sus respectivos pesos) y el sesgo, y decide la señal de salida de la neurona.

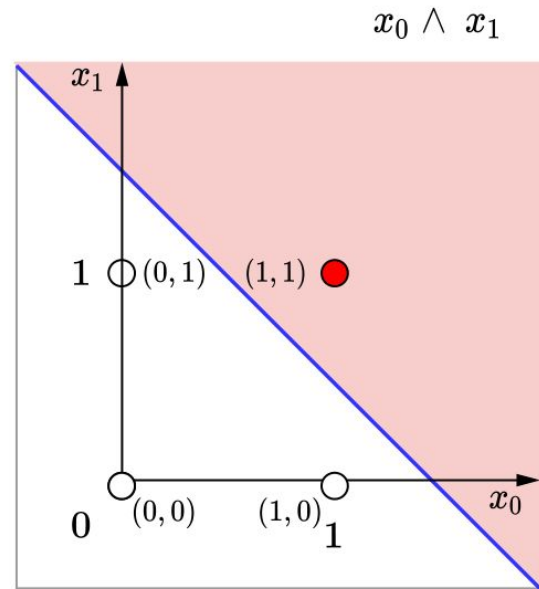
Función Específica: Históricamente, el Perceptrón original utilizaba una simple **función escalón** (*step function*) o una **función umbral**. Esta función da una salida binaria (por ejemplo, 1 o 0, o 1 o -1) dependiendo de si la suma ponderada supera o no un umbral predefinido.

Perceptron AND



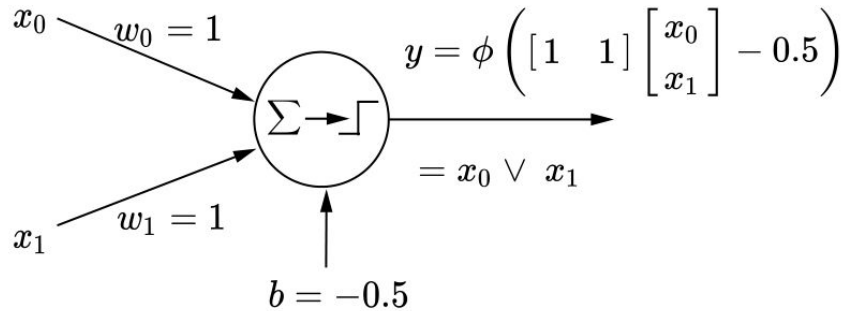
(a) Perceptron for a logical AND function:
 $y = x_0 \wedge x_1$

Una línea recta puede separar los dos conjuntos de puntos.



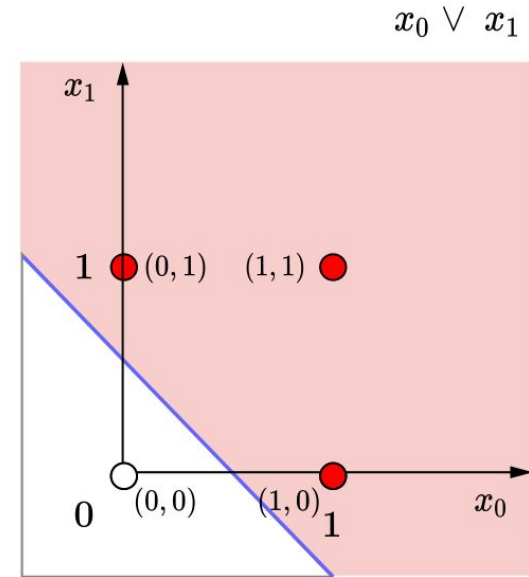
(a) Logical AND
decision boundary:
 $x_0 + x_1 = 1.5$

Perceptron OR



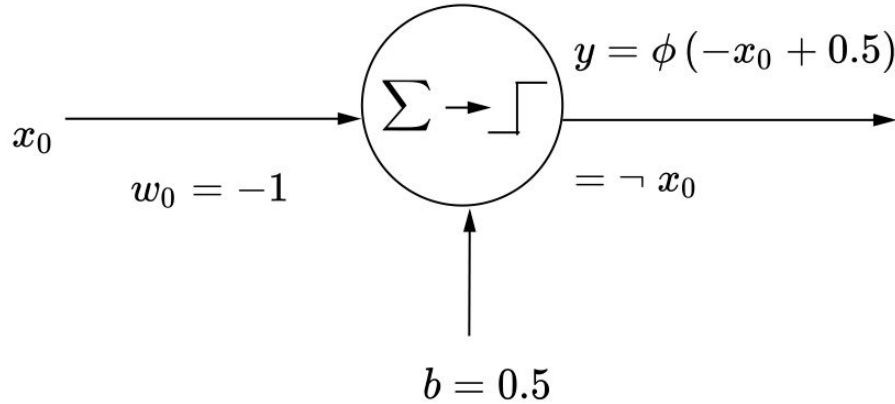
(b) Perceptron for a logical OR function:
 $y = x_0 \vee x_1$

Una línea recta puede separar los dos conjuntos de puntos.



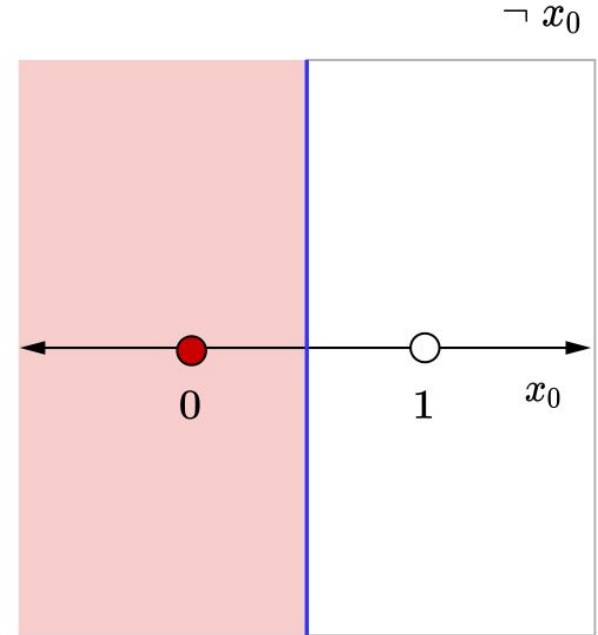
(b) Logical OR
decision boundary:
 $x_0 + x_1 = 0.5$

Perceptron NOT



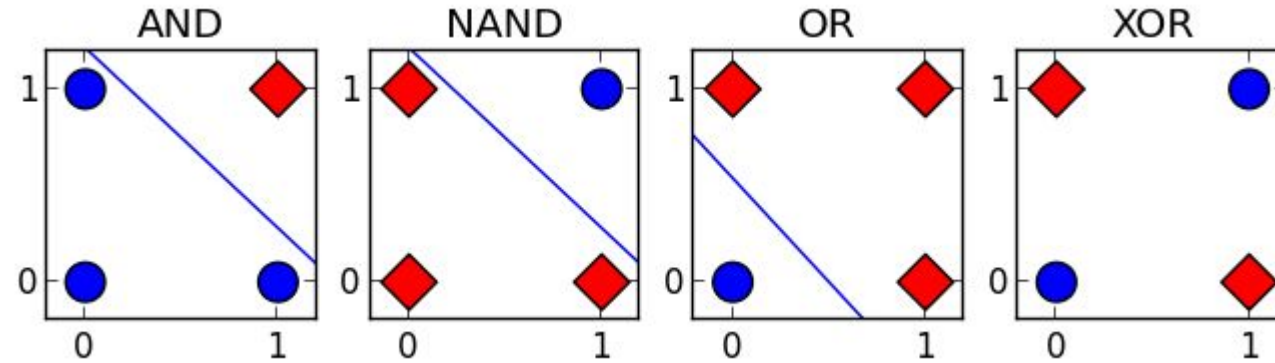
(c) Perceptron for a logical NOT function:
 $y = \neg x_0$

Una línea recta puede separar los dos conjuntos de puntos.



(c) Logical NOT
decision boundary:
 $x_0 = 0.5$

Problema del XOR

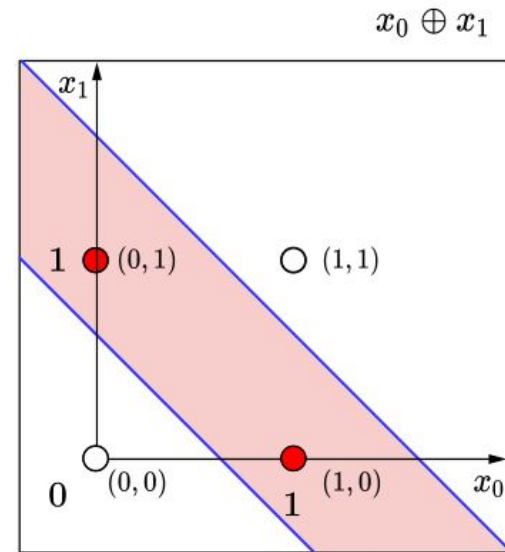
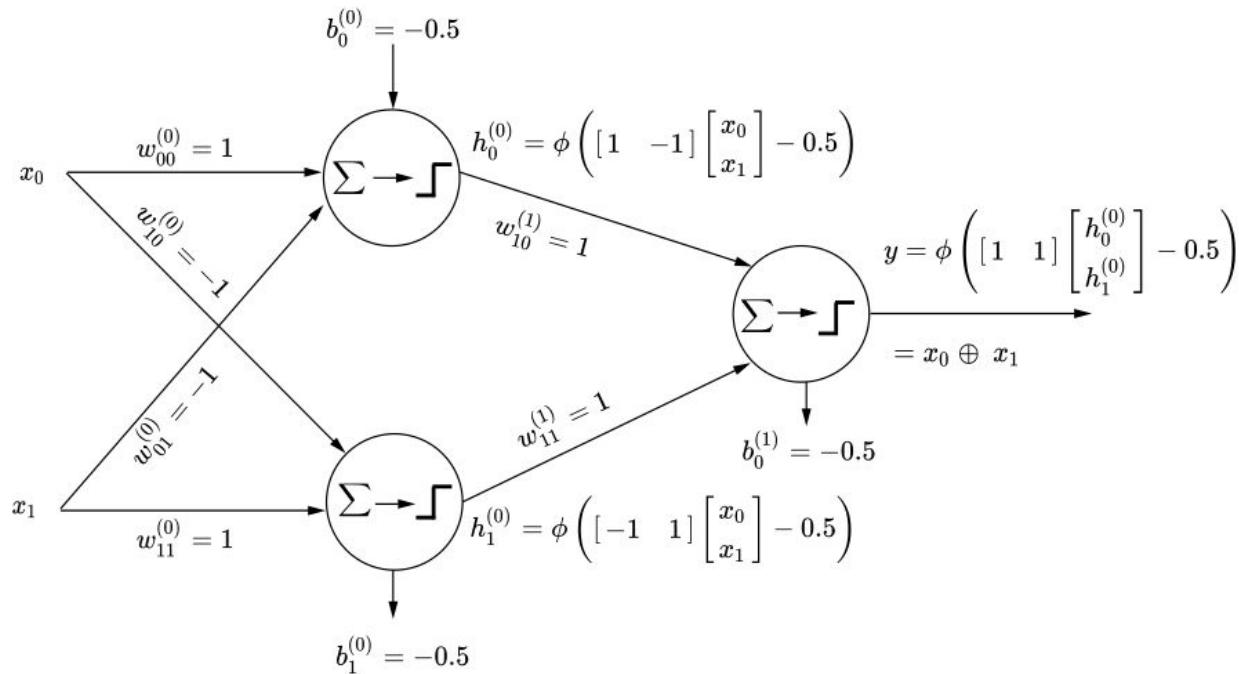


A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Un perceptrón simple toma decisiones basándose en una suma ponderada de entradas, lo que significa que funciona estrictamente como un **clasificador lineal**. Su forma de clasificar datos es equivalente a trazar una única línea recta (o un hiperplano) en un gráfico bidimensional para separar diferentes grupos.

Es geoméricamente imposible trazar una sola línea recta que deje a los puntos que dan "1" de un lado y a los que dan "0" del otro. Al ser matemáticamente imposible separar estos datos linealmente, un perceptrón simple es incapaz de modelar la función XOR

Solución al XOR



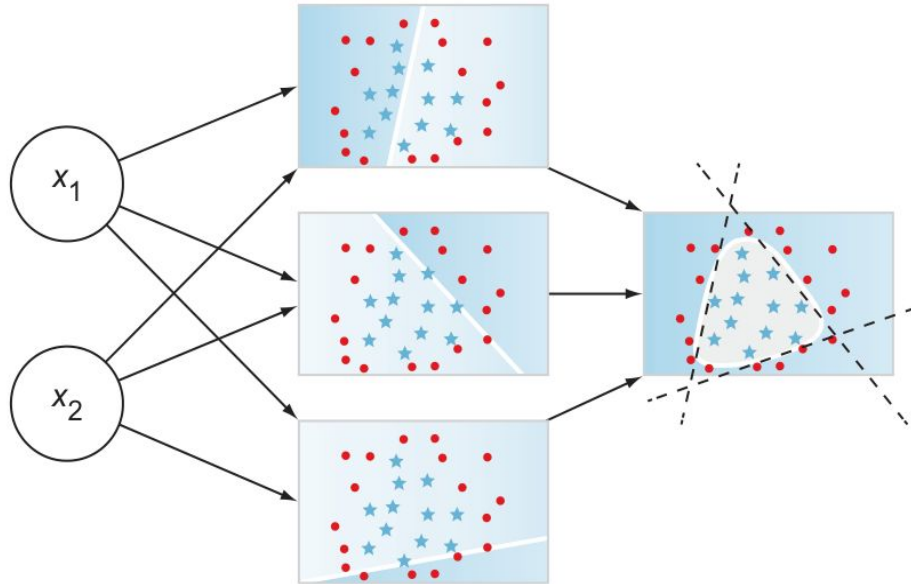
Son necesarias dos líneas y para un perceptrón simple es imposible

Limitaciones del perceptrón

Input features

Hidden layer

Output

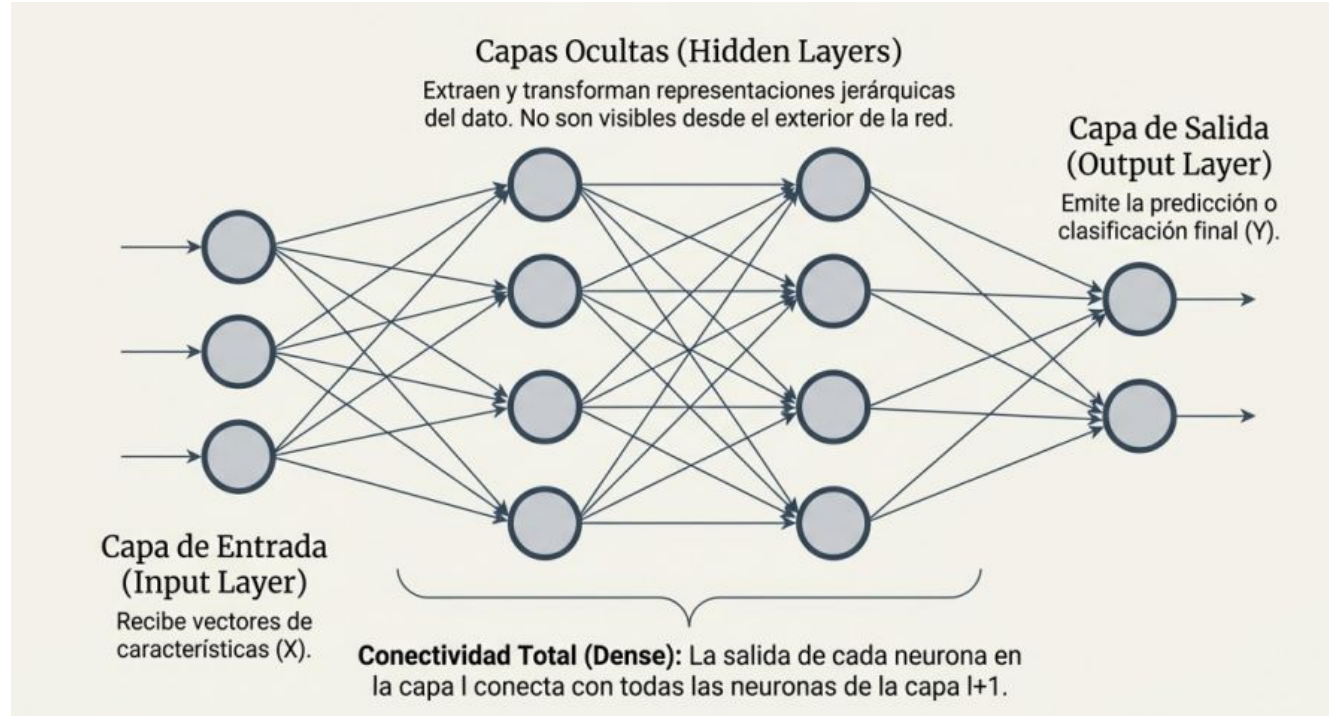


El perceptrón de Rosenblatt, como modelo fundacional de red neuronal, está inherentemente limitado a aprender solo fronteras de decisión lineales. La imagen muestra que la unidad es capaz de trazar una única 'línea recta' (un hiperplano en dimensiones superiores) para separar dos clases de datos. Si los datos no son linealmente separables (como el problema XOR), un perceptrón simple no puede encontrar esta 'línea recta' y, por lo tanto, no puede aprender la clasificación correcta.

Perceptrón de varias capas (MLP)

Multilayer perceptron (MLP)

Un Perceptrón Multicapa (MLP) es un tipo de red neuronal artificial *feedforward* (propagación hacia adelante). Su arquitectura se define por la presencia de múltiples capas de neuronas organizadas en tres partes principales.



Capa de entrada

La capa de entrada es la interfaz entre el mundo exterior y la red neuronal. Su función es recibir las características del conjunto de datos y transmitirlos a las capas subsiguientes. El número de neuronas en esta capa es igual al número de variables predictoras o características presentes en los datos brutos.

Por ejemplo, en una red diseñada para clasificar imágenes de 28x28 píxeles, la capa de entrada contendría 784 neuronas, cada una representando la intensidad lumínica de un píxel individual.

Capas ocultas (hidden layers)

Las capas ocultas son el "motor" de aprendizaje de la red. Se denominan "ocultas" porque sus entradas y salidas no son visibles desde el exterior del sistema. Una red se considera "profunda" si posee al menos dos capas ocultas. En estas capas, las neuronas realizan transformaciones matemáticas sucesivas que permiten extraer representaciones abstractas de los datos. Mientras que las primeras capas ocultas pueden detectar patrones simples (como bordes en una imagen), las capas más profundas combinan estos patrones para reconocer conceptos complejos (como formas o rostros). Cada neurona en una capa oculta está conectada a todas las neuronas de la capa anterior a través de pesos, y su salida se determina tras aplicar la función de activación elegida.

Capa de salida

El aprendizaje en una red neuronal es un proceso cíclico compuesto por dos fases: la propagación hacia adelante (*forward propagation*) y la propagación hacia atrás (*backpropagation*). La propagación hacia adelante es el cálculo determinista que transforma la entrada en una predicción. En el contexto del aprendizaje profundo moderno, este flujo no se realiza valor por valor, sino mediante el procesamiento masivo de **tensores**.

Pesos o weights

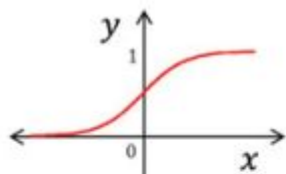
Son los enlaces que unen las neuronas de una capa con las de la siguiente. A cada conexión se le asigna un valor numérico o "peso" (weight) que refleja la importancia de esa señal de entrada en el proceso de toma de decisiones para la predicción final.

Las **funciones de activación** son componentes determinantes que transforman la suma ponderada de las entradas de una neurona en una señal de salida que se propaga a la siguiente capa.

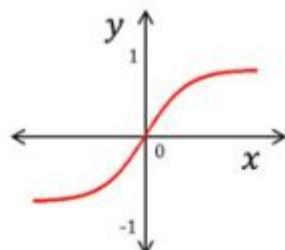
Su importancia es crítica, ya que:

- **Introducen la No Linealidad:** Sin una función de activación no lineal, una red neuronal de múltiples capas se reduciría matemáticamente a una simple regresión lineal, lo que invalidaría el beneficio de la profundidad arquitectónica.
- **Permiten Capturar Complejidad:** La introducción de la no linealidad permite a la red deformar el espacio de características, facilitando la captura de relaciones extremadamente complejas y fronteras de decisión curvas.

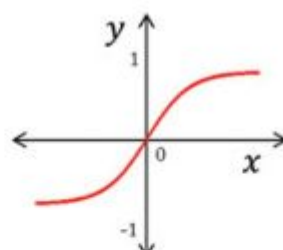
Algunas funciones de activación



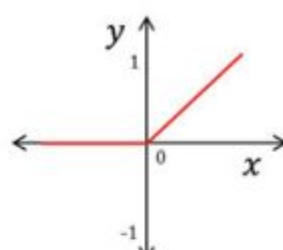
(a) Sigmoid



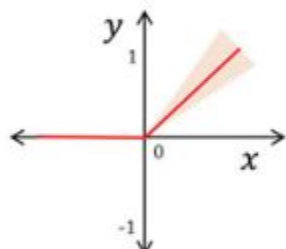
(b) Tanh



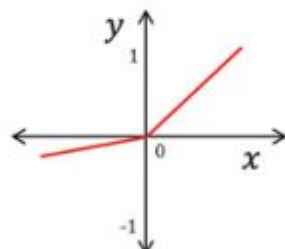
(c) Algebraic Sigmoid



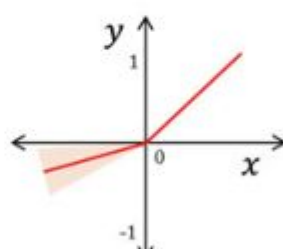
(d) ReLU



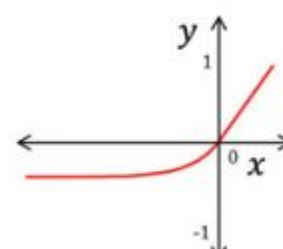
(e) Noisy ReLU



(f) Leaky ReLU/PreLU



(g) Randomized
Leaky ReLU

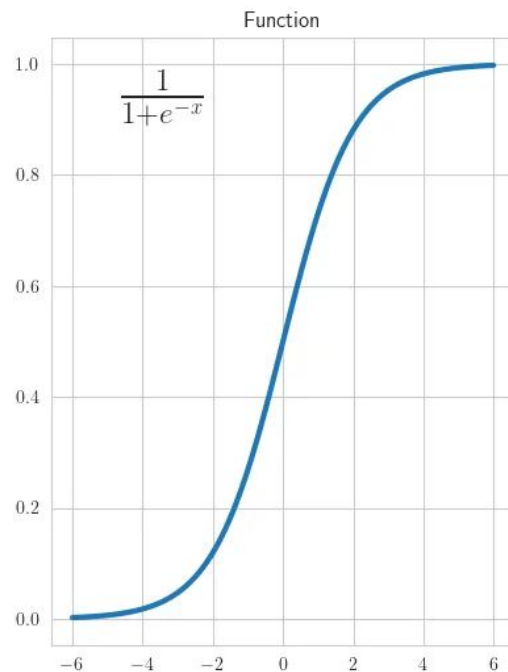


(h) Exponential
Linear Unit

Figure 4.9: Some of the common activation functions that are used in deep neural networks.

sigmoid

La función **Sigmoide** fue la primera en ganar popularidad masiva tras el perceptrón de Rosenblatt. Transforma cualquier valor real en el intervalo abierto (0, 1), lo que la hace ideal para interpretar la salida de una neurona como una probabilidad. Su forma suave de "S" permite que sea diferenciable en todos sus puntos, un requisito indispensable para el algoritmo de backpropagation. Sin embargo, la sigmoide sufre de dos problemas críticos: no está centrada en cero, lo que puede causar actualizaciones de pesos sesgadas, y presenta el fenómeno de saturación. Cuando las entradas son muy grandes o muy pequeñas, la pendiente de la función se vuelve casi plana (gradiente cercano a cero), lo que detiene el aprendizaje en las capas profundas al "desvanecer" el gradiente.



Para ver su derivada:

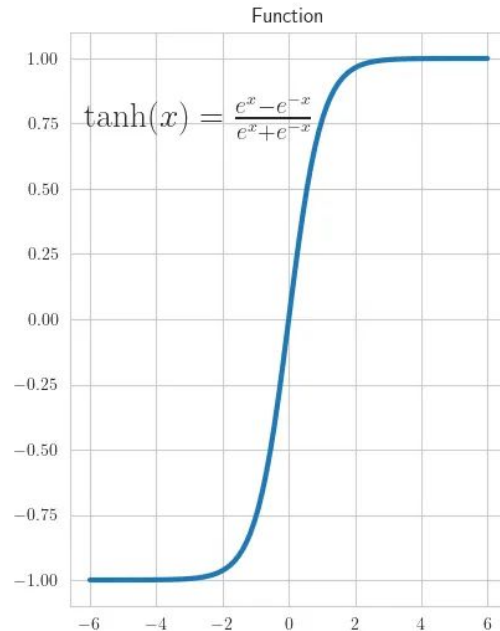
<https://ml-explained.com/blog/activation-functions-explained#sigmoid-function>

$$f(x) = \frac{1}{1 + e^{-x}}$$

tanh

La **Tangente Hiperbólica (Tanh)** surgió como una mejora directa sobre la sigmoide. **Al estar centrada en cero y tener un rango de (-1, 1), las activaciones de las neuronas pueden ser tanto positivas como negativas, lo que facilita que los promedios de las actualizaciones de pesos se acerquen a cero, acelerando la convergencia.**

A pesar de esta ventaja, la Tanh también presenta saturación en sus extremos, heredando parcialmente el problema del gradiente desvaneciente en redes muy profundas.



Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#tanh-function>

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

La función de activación **Softmax** (también conocida como función exponencial normalizada) es una herramienta fundamental en las redes neuronales, utilizada casi exclusivamente en la **capa de salida para problemas de clasificación multiclase**.

¿Cómo funciona? Toma un vector de puntuaciones brutas (conocidas como *logits*, que pueden ser números positivos, negativos o muy grandes) y lo transforma en una **distribución de probabilidad**. Matemáticamente, aplica una función exponencial (e^x) a cada valor individual para garantizar que todos se vuelvan positivos, y luego divide cada uno por la suma total de estos valores exponenciales.

Este cálculo asegura dos propiedades vitales para la red: **todas las activaciones resultantes estarán entre 0 y 1, y la suma conjunta de todas las salidas será exactamente 1**

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

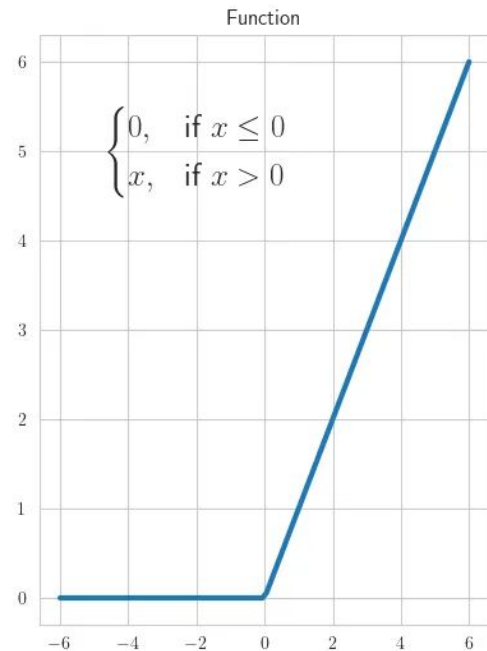
RELU

La **función de activación ReLU** ha revolucionado el aprendizaje profundo moderno. Definida como $f(x) = \max(0, x)$, es computacionalmente eficiente ya que no requiere el cálculo de exponenciales costosas, limitándose a una operación de umbralización simple.

Su mayor virtud es que no se satura para valores positivos; la derivada es constantemente 1 para cualquier $x > 0$, lo que permite que el gradiente fluya sin atenuarse a través de decenas de capas. Además, ReLU promueve la "activación dispersa" (*sparsity*), ya que solo una fracción de las neuronas se activa en un momento dado, lo que imita la eficiencia energética del cerebro biológico y mejora la capacidad de generalización del modelo.

Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#rectified-linear-unit-relu>

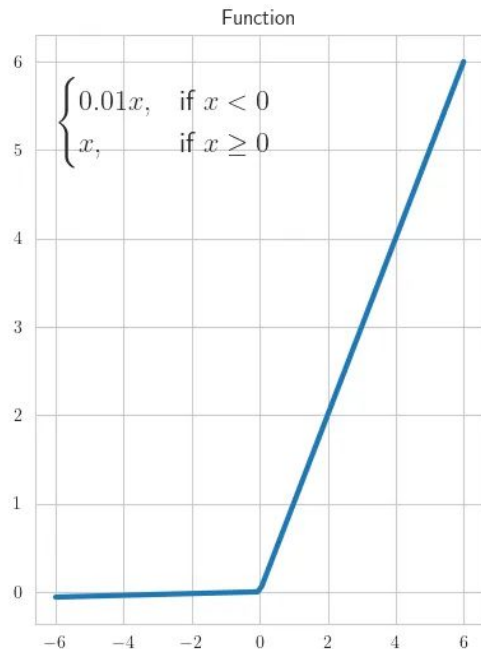


$$f(x) = \max(0, x)$$

Leaky RELU

La función de activación **Leaky ReLU** (Rectificadora Lineal Unitaria con "fugas" o "pérdidas") es una generalización o variante de la popular función ReLU, diseñada para superar algunas de las limitaciones matemáticas de ReLU.

A diferencia de la función ReLU estándar, que devuelve exactamente cero cuando la entrada es negativa, **Leaky ReLU introduce una pendiente muy pequeña (típicamente de 0.01) para los valores negativos**. Para los valores positivos, se comporta de forma idéntica a la ReLU normal y pasa la señal de entrada sin cambios.



Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#leaky-relu>

$$f(x) = \begin{cases} 0.01x, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

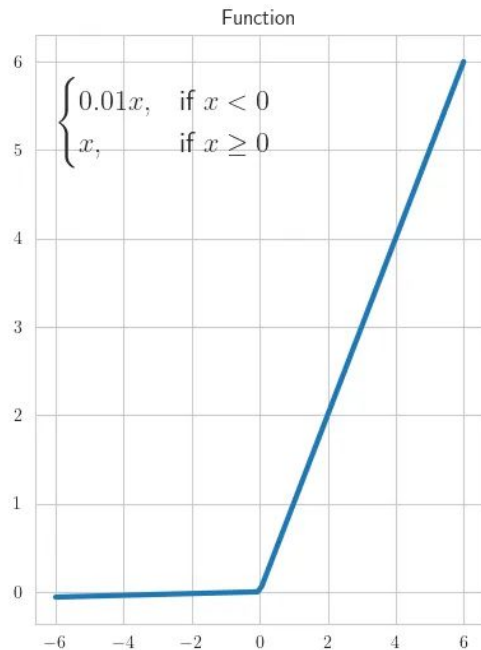
Leaky RELU

Leaky ReLU se creó principalmente para mitigar un fenómeno conocido como el problema de las "neuronas muertas" o *dying ReLU*. En la función ReLU normal, si el valor de entrada es menor que cero, la salida es cero y, por ende, su derivada también es exactamente cero. Durante el proceso de entrenamiento de la red (mediante el algoritmo de *backpropagation* o retropropagación), si el gradiente es cero, los pesos asociados a esa neurona no pueden actualizarse. Esto significa que si las neuronas reciben consistentemente valores negativos, se vuelven inactivas y dejan de aprender.

Al agregar la pequeña pendiente de 0.01 para los números negativos, **Leaky ReLU garantiza que la derivada nunca sea completamente cero**. De esta forma, siempre habrá un pequeño gradiente que fluya a través de la neurona, permitiendo que siga ajustando sus pesos y aprendiendo, incluso si sus niveles de activación caen por debajo de cero.

Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#leaky-relu>



$$f(x) = \begin{cases} 0.01x, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

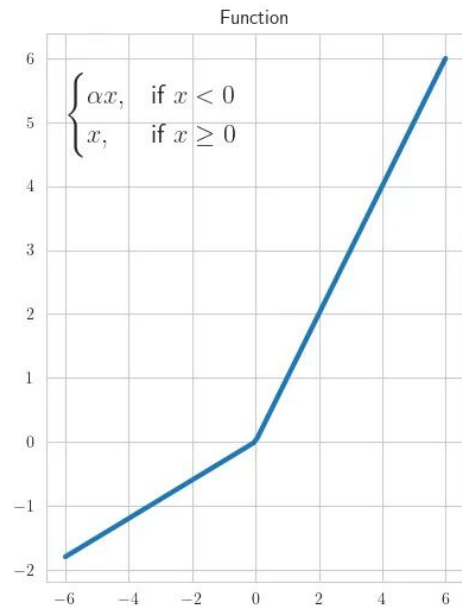
Parametric ReLU

La **Parametric ReLU (PReLU)**, o Unidad Lineal Rectificada Paramétrica, es una variante de la función de activación ReLU y una evolución directa de la Leaky ReLU.

Al igual que la Leaky ReLU, su objetivo es solucionar el problema de las "neuronas muertas" permitiendo que pase una pequeña señal cuando el valor de entrada es negativo, en lugar de bloquearlo convirtiéndolo en cero .

La diferencia clave frente a Leaky ReLU es que el factor a (la pendiente de la parte negativa) no es una constante fija predefinida por el programador. En su lugar, a **es un parámetro aprendible**. Esto significa que la red neuronal descubre, adapta y optimiza automáticamente la pendiente ideal a partir de los datos durante la fase de entrenamiento mediante retropropagación.

Para ver su derivada: <https://ml-explained.com/blog/activation-functions-explained#parametric-relu>



$$f(x) = \begin{cases} \alpha x, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

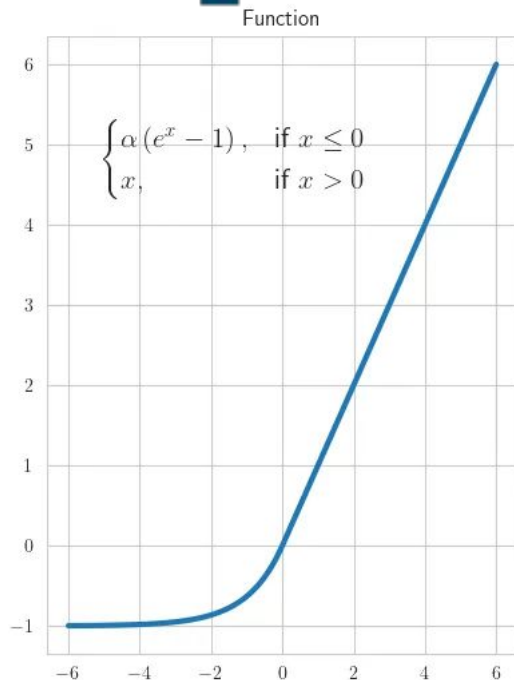
ELU

Propósito: Al igual que la función Leaky ReLU que mencionamos anteriormente, la función ELU fue investigada como un intento de **resolver el problema de las "neuronas muertas"** (*dying neuron problem*), que ocurre cuando una neurona solo recibe valores negativos, su gradiente se vuelve plano y pierde la capacidad de actualizar sus pesos para seguir aprendiendo.

Comportamiento: A diferencia de la función ReLU estándar (que tiene un corte abrupto en cero), ELU forma parte de un grupo de **funciones suaves** (*smooth functions*) diseñadas para limitar el gradiente cuando los valores de entrada son negativos, permitiendo así que siga habiendo flujo de información.

Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#exponential-linear-unit-elu>



$$f(x) = \begin{cases} \alpha (e^x - 1), & \text{if } x \leq 0 \\ x, & \text{if } x > 0 \end{cases}$$

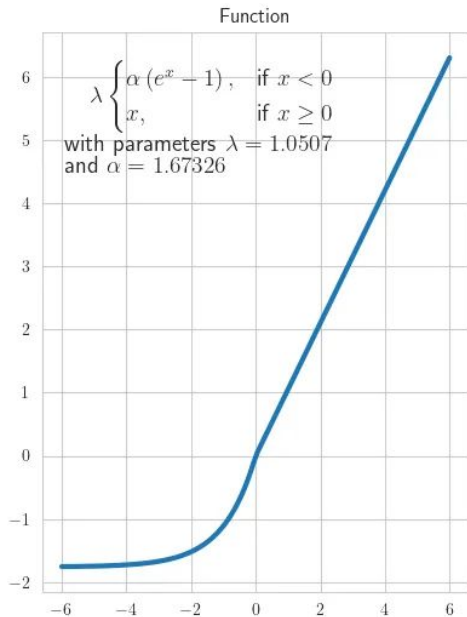
SELU

La característica que distingue a SELU de otras funciones es que posee la propiedad de **auto-normalización**: bajo ciertas condiciones, las activaciones de cada capa convergen automáticamente hacia **media 0 y varianza 1** a medida que la señal se propaga hacia adelante en la red.

Formalmente, SELU define un **mapeo contractivo** en el espacio de medias y varianzas

Esto evita los problemas de **vanishing y exploding gradients** sin necesidad de Batch Normalization.

$$f(x) = \lambda \begin{cases} \alpha (e^x - 1), & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases} \text{with parameters } \lambda = 1.0507 \text{ and } \alpha = 1.67326$$



Para ver su derivada:

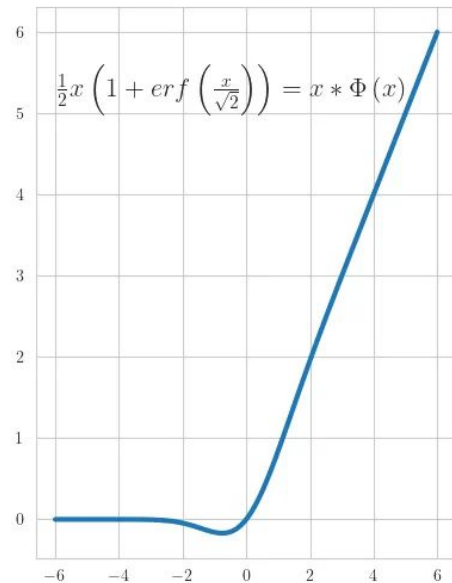
<https://ml-explained.com/blog/activation-functions-explained#scaled-exponential-linear-unit-selu>

GELU

GELU (Gaussian Error Linear Unit, o Unidad Lineal de Error Gaussiano) es una función de activación de alto rendimiento que se ha convertido en el estándar por defecto para arquitecturas profundas modernas, especialmente en modelos de lenguaje basados en Transformers como BERT y GPT.

A diferencia de funciones más antiguas como ReLU, **GELU pondera o escala los valores de entrada basándose en su magnitud mediante un enfoque probabilístico y suave.**

Curva completamente suave y no monótona: GELU no tiene un "quiebre" en el origen como ReLU. De hecho, tiene una pequeña caída hacia valores negativos para entradas ligeramente por debajo de cero.



$$f(x) = \frac{1}{2}x \left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right) = x * \Phi(x)$$

Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#gaussian-error-linear-unit-gelu>

SiLU / Swish

SiLU (Sigmoid Linear Unit), también conocida como **Swish**, es una función de activación suave y no monótona que se ha consolidado como el estándar principal para las arquitecturas modernas de modelos de lenguaje decodificadores (como LLaMA, Mistral y GPT-NeoX).

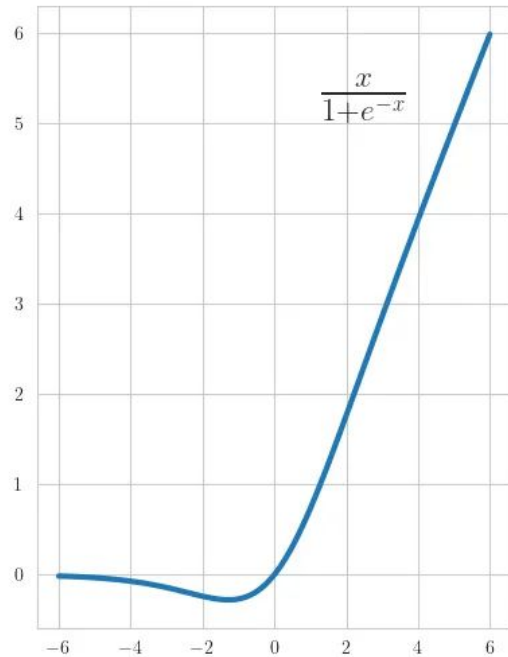
Al igual que la función GELU, SiLU carece del quiebre o "corte brusco" en cero que caracteriza a ReLU.

Aunque sus formas son parecidas, SiLU es computacionalmente más simple que la versión exacta de GELU, ya que solo requiere el cálculo de una función sigmoide y una multiplicación.

Mientras que GELU se estableció como la norma para los modelos "codificadores" clásicos (como BERT), **SiLU dominó la nueva ola de Modelos de Lenguaje Grandes (LLMs)**

Para ver su derivada:

<https://ml-explained.com/blog/activation-functions-explained#sigmoid-weighted-linear-unit-silu--swish>



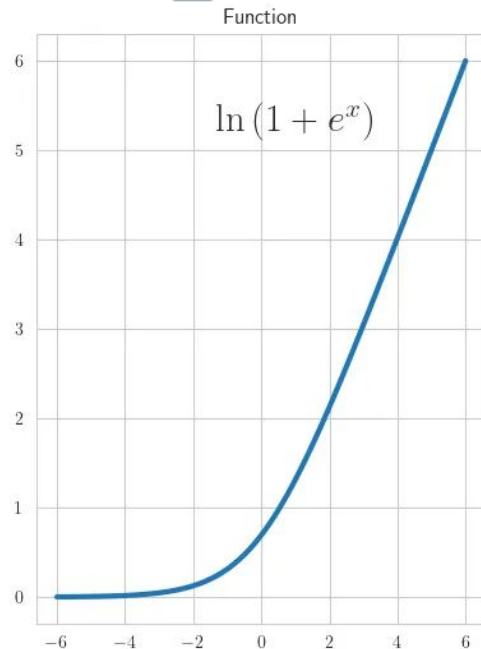
$$f(x) = \frac{x}{1+e^{-x}}$$

Softplus es una función de activación suave que actúa como una aproximación diferenciable de la función ReLU. Su objetivo es mantener los beneficios principales de ReLU, pero resolviendo el problema de las "neuronas muertas" mediante transiciones matemáticas continuas.

Para valores positivos grandes, Softplus devuelve valores casi idénticos a la entrada (al igual que ReLU). Sin embargo, la diferencia clave radica en los valores negativos: en lugar de devolver un cero exacto, Softplus devuelve valores positivos muy pequeños .

Ventajas:

- Suavidad y diferenciabilidad infinita
- Prevención de neuronas muertas
- Salidas estrictamente positivas



$$f(x) = \ln(1 + e^x)$$

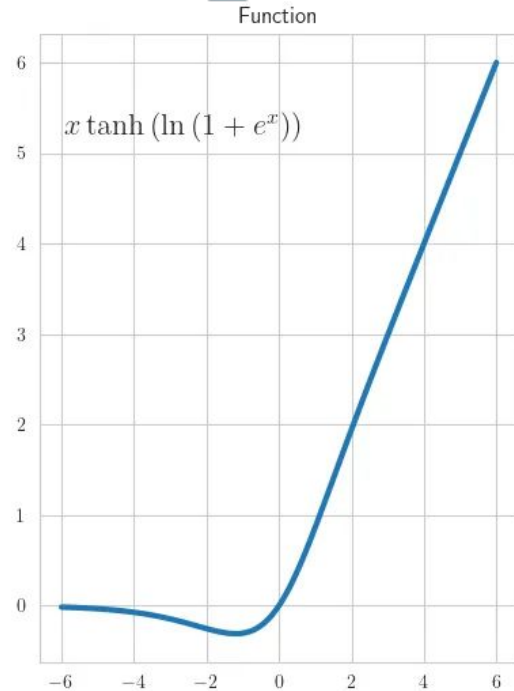
Para ver su derivada: <https://ml-explained.com/blog/activation-functions-explained#softplus>

Mish es una función de activación no monótona, suave y autorregularizada, estrechamente relacionada con Swish y GELU.

Mish actúa modulando la entrada basándose en sus propiedades, pero agrega beneficios específicos a la dinámica de entrenamiento de la red.

Perfil suave y continuamente diferenciable: A diferencia de ReLU, Mish no tiene quiebres bruscos. Esta suavidad continua evita singularidades matemáticas y efectos secundarios no deseados al optimizar la red basándose en los gradientes

Efecto de "precondicionador" (Paisaje de pérdida más suave): Se hipotetiza que el comportamiento de la primera derivada de Mish actúa como un precondicionador matemático. En la práctica, esto significa que **Mish transforma el panorama de error (loss landscape) en uno mucho más suave.**

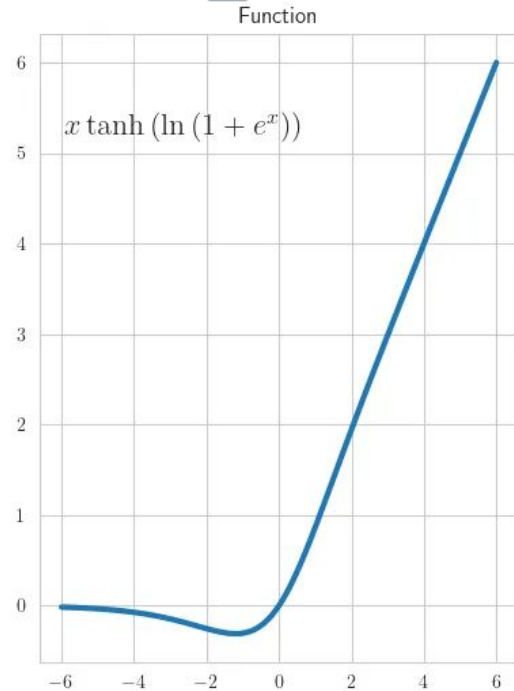


$$f(x) = x \tanh(\ln(1 + e^x))$$

Para ver su derivada: <https://ml-explained.com/blog/activation-functions-explained#mish>

Acotada por abajo y no acotada por arriba: Al no tener límite hacia los valores positivos, evita que la red se sature (lo cual frena el entrenamiento por tener gradientes cercanos a cero). Al tener un límite inferior (una pequeña caída cóncava para los valores negativos antes de tender a cero), produce un fuerte efecto de regularización y **preserva una pequeña cantidad de información negativa**, eliminando por diseño el problema de las "neuronas muertas" que sufre ReLU

Estabilidad en redes muy profundas: Mientras que el rendimiento de funciones como ReLU o Swish tiende a caer en picada cuando se construyen modelos con decenas de capas (por ejemplo, arquitecturas de visión de más de 15 capas), Mish es capaz de mantener una alta precisión y estabilidad frente a arquitecturas sumamente complejas . En computer vision, mish supera a ReLU, Leaky ReLU y Swish.



$$f(x) = x \tanh(\ln(1 + e^x))$$

Para ver su derivada: <https://ml-explained.com/blog/activation-functions-explained#mish>

Resumiendo

Función de activación	Concepto	¿Qué agrega a la red? (Ventajas)	Limitaciones y Casos de Uso
sigmoid	Comprime los valores entre 0 y 1	Produce una curva suave. Sus salidas se pueden interpretar como probabilidades, lo que la hace ideal como capa final para problemas de clasificación binaria	Sufre del problema de desvanecimiento del gradiente en redes profundas, ya que en los valores muy grandes o muy pequeños la pendiente es casi cero y la red deja de aprender
tanh	Comprime entre -1 y 1	Está centrada en cero , lo que ayuda a que el algoritmo de aprendizaje converja más rápido que con la sigmoide. Sus gradientes cerca de cero son más fuertes	Al igual que la sigmoide, satura en los extremos y también sufre de desvanecimiento del gradiente
softmax	Normaliza un vector	Toma puntuaciones (<i>logits</i>) y garantiza que se conviertan en una distribución de probabilidad donde la suma de todos los elementos es exactamente 1	Sensible a valores atípicos (amplifica diferencias pequeñas). Es la opción estándar exclusivamente para la capa final en modelos de clasificación multiclase

Resumiendo

Función de activación	Concepto	¿Qué agrega a la red? (Ventajas)	Limitaciones y Casos de Uso
ReLU	Deja pasar positivos, hace 0 los negativos	Previene el desvanecimiento del gradiente para valores positivos. Promueve activaciones dispersas (cómputo eficiente, ignorando negativos) permitiendo entrenar redes muy profundas	Problema de " neuronas muertas ": si un valor ingresa constantemente en negativo, su gradiente será 0 y la neurona se apagará para siempre. No es diferenciable exactamente en 0.
Leaky ReLU	Fuga pequeña para negativos	Resuelve el problema de las neuronas muertas de ReLU permitiendo un flujo de gradiente pequeño y constante (como 0.01) para valores negativos	Aunque mitiga el problema de las neuronas muertas, todavía tiene un "quiebre" abrupto en el cero, careciendo de una transición suave
Parametric ReLU	Idéntica a Leaky ReLU, pero la pendiente de la parte negativa (a) no es fija, sino que es un parámetro que la red aprende de los datos durante el entrenamiento	Resuelve el problema de las "neuronas muertas". Al adaptar la pendiente a los datos, mejora la precisión frente a ReLU, habiendo sido un factor clave para superar el rendimiento a nivel humano en tareas complejas de clasificación como ImageNet	Añade complejidad al modelo; el entrenamiento de este parámetro exige tiempo y técnicas cuidadosas de regularización. Además, es muy propensa a sufrir sobreajuste (overfitting) si se entrena en conjuntos de datos pequeños

Resumiendo

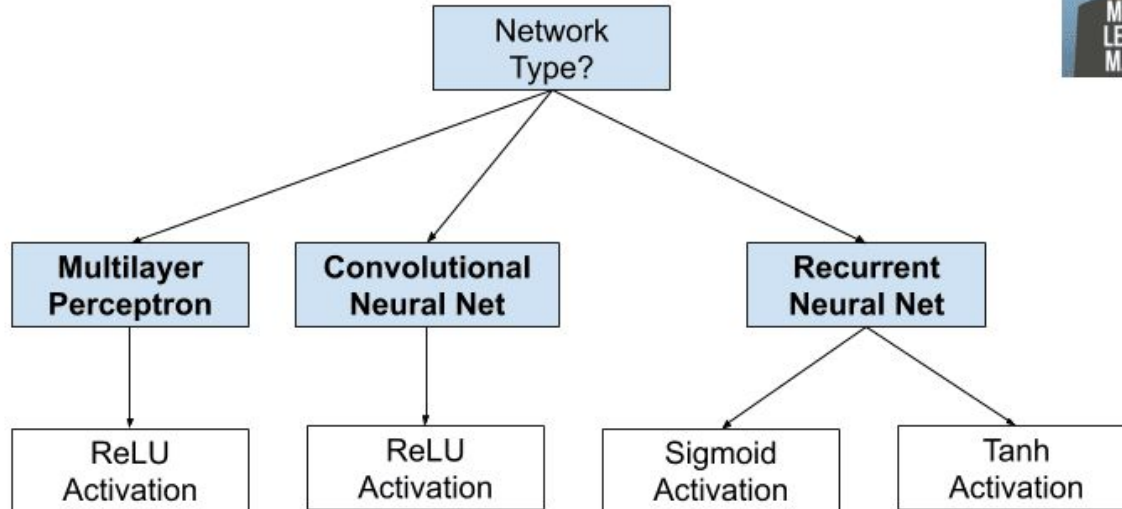
Función de activación	Concepto	¿Qué agrega a la red? (Ventajas)	Limitaciones y Casos de Uso
ELU		Tiene valores negativos suaves (mediante una función exponencial), lo que ayuda a empujar la media de activaciones hacia cero y acelera la convergencia	Calcular la función exponencial es computacionalmente más costoso que el simple umbral de la función ReLU
SELU		Permite construir Redes Neuronales Auto-Normalizadas (<i>Self-Normalizing Neural Networks</i> de Klambauer et al.)	Se usa típicamente en arquitecturas profundas junto a capas de incrustación densamente conectadas para ayudar al modelo a mantener automáticamente la distribución de sus variables
GELU		Actúa como una compuerta probabilística y completamente suave . Evita el quiebre abrupto de ReLU y es el estándar absoluto en Transformers codificadores como BERT o GPT	Es matemáticamente más costosa de calcular, por lo que en la práctica se suele usar una aproximación rápida basada en Tanh (<i>tanh approximation</i>)

Resumiendo

Función de activación	Concepto	¿Qué agrega a la red? (Ventajas)	Limitaciones y Casos de Uso
SiLU (Swish)		Es suave y no monótona. A diferencia de GELU, su derivada puede superar el valor de 1 (alrededor de 1.1), lo que ayuda a amplificar ligeramente la señal de gradiente en redes muy profundas	Es el estándar dominante en Transformers decodificadores modernos (como LLaMA y Mistral). Suele ser más rápida de calcular que la función GELU exacta
softplus	Es una aproximación suave y continuamente diferenciable de ReLU.	Su derivada es continua (es la función sigmoide), lo que estabiliza el flujo del gradiente. Evita las neuronas muertas ya que no devuelve ceros exactos ante entradas negativas. Como siempre produce resultados positivos, es ideal para modelos bayesianos, probabilísticos y de regresión.	Tiene un mayor costo computacional que ReLU debido al cálculo de logaritmos y exponenciales. Al no tener activaciones en cero absoluto, carece de dispersión (<i>sparsity</i>), lo que puede hacer la convergencia más lenta en redes profundas. Requiere el uso de umbrales en el código para evitar desbordamientos numéricos con valores grandes.
mish	Función suave, no monótona y autorregularizada	Funciona como un "precondicionador" matemático que suaviza el paisaje de pérdida (loss landscape) , creando mínimos más amplios y estables	Su principal desventaja es un alto costo computacional , ya que los cálculos matemáticos para sus pasadas hacia adelante y hacia atrás son pesados y lentos.

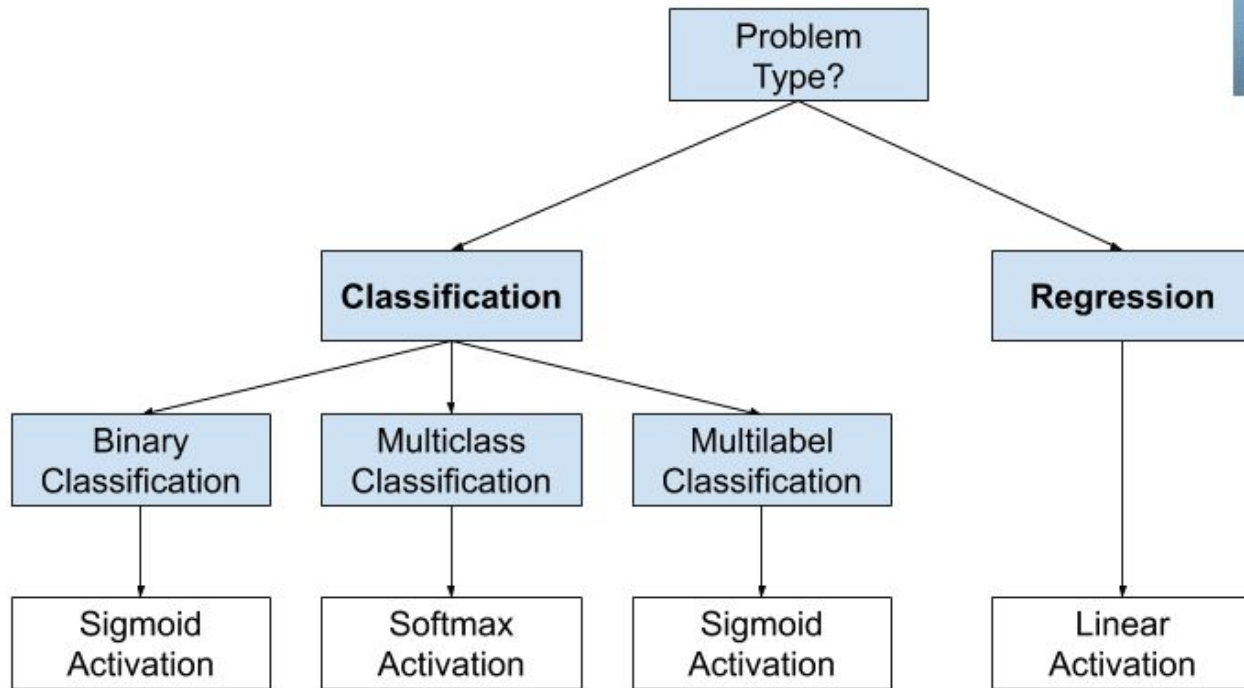
Cómo elegir una función de activación en una capa oculta

How to Choose an Hidden Layer Activation Function

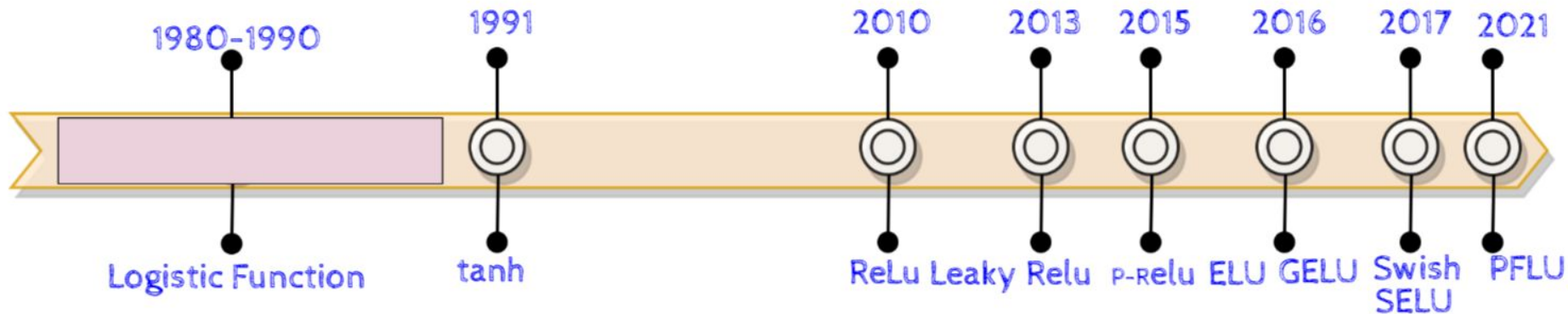


Cómo elegir una función de activación en una capa final

How to Choose an Output Layer Activation Function



Desarrollo de funciones de activación



Funciones de pérdida (loss functions)

Una **función de pérdida** (también conocida como función objetivo o función de coste) es una herramienta matemática que mide qué tan lejos está la predicción de una red neuronal del resultado real esperado (conocido como *ground truth*).

El propósito principal de esta función es guiar el aprendizaje automatizado de la red. Al calcular el gradiente (la derivada) de la función de pérdida, los algoritmos de optimización —como el descenso de gradiente— pueden determinar exactamente cómo ajustar los pesos y sesgos del modelo para reducir dicho error en las siguientes iteraciones.

Las más utilizadas en el aprendizaje profundo son:

- **Mean Squared Error (MSE) / Error Cuadrático Medio.**
- **Binary Cross-Entropy (BCE) / Entropía Cruzada Binaria:** Se utiliza principalmente en problemas de **clasificación binaria** (decisiones de sí/no o clase A/clase B). También es la opción predeterminada para problemas de **clasificación multietiqueta**

Funciones de pérdida (loss functions)

- **Cross-Entropy Loss (Entropía Cruzada):** Es la función por excelencia para tareas de **clasificación multiclase** de una sola etiqueta (por ejemplo, determinar si en una imagen hay un perro, un gato o un avión). Mide la distancia matemática entre la distribución de probabilidades que genera el modelo y la distribución real de las categorías.
- **Focal Loss (Pérdida Focal):** Es una evolución de la entropía cruzada diseñada específicamente para resolver problemas con **desequilibrio de datos** (cuando tienes muchísimas muestras de una categoría y muy pocas de otra). Modifica el cálculo de pérdida para quitarle peso a los ejemplos "fáciles" que el modelo ya clasifica bien, forzando a la red a concentrar su aprendizaje en aquellos ejemplos "difíciles" que aún no logra comprender .
- **Hinge Loss:** Inspirada en el comportamiento de las Máquinas de Vectores de Soporte (SVM), penaliza las predicciones a menos que la puntuación de la clase correcta sea superior a la de las incorrectas por un "margen" predefinido. A diferencia de la entropía cruzada, una vez que la red supera ese margen de seguridad, la pérdida se vuelve exactamente cero y deja de intentar seguir separando las puntuaciones de forma infinita.

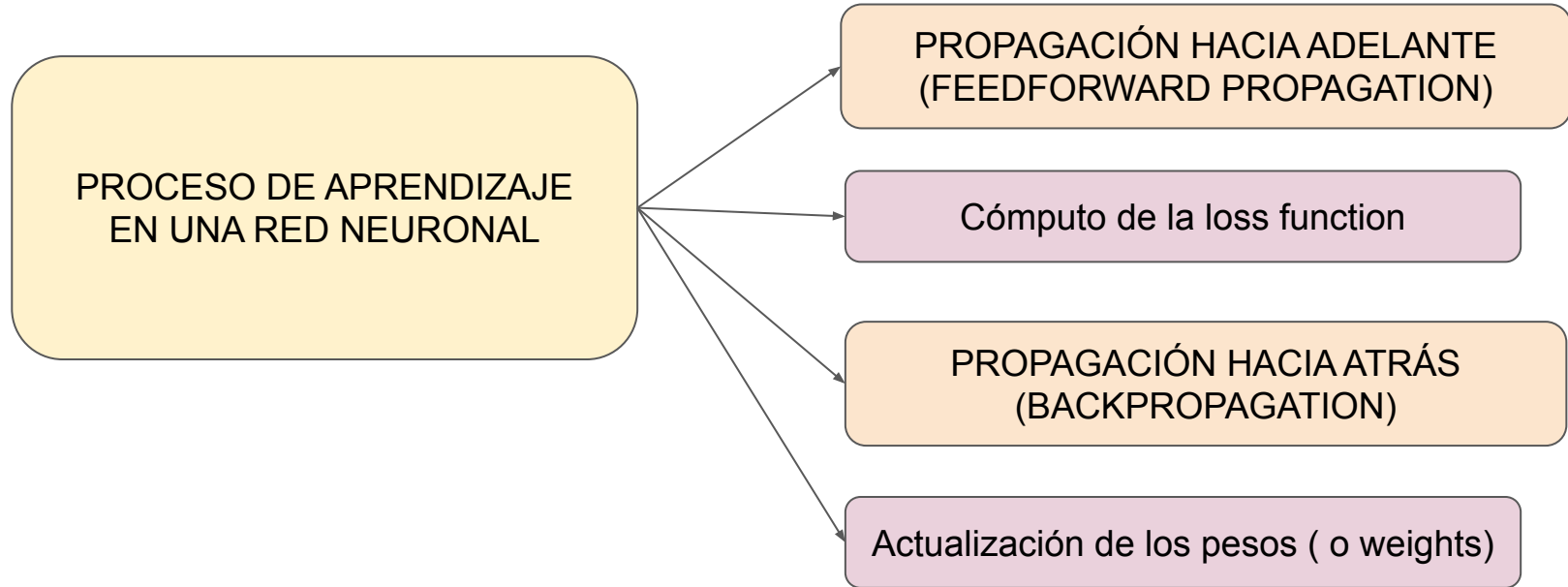
Loss functions para regresión

Mean Bias Error	Captures average bias in prediction. But is rarely used for training.	$\mathcal{L}_{\mathcal{MBE}} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$
Mean Absolute Error	Measures absolute average bias in prediction. Also called L1 Loss.	$\mathcal{L}_{\mathcal{MAE}} = \frac{1}{N} \sum_{i=1}^N y_i - f(x_i) $
Mean Squared Error	Average squared distance between actual and predicted. Also called L2 Loss.	$\mathcal{L}_{\mathcal{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$
Root Mean Squared Error	Square root of MSE. Loss and dependent variable have same units.	$\mathcal{L}_{\mathcal{RMSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$
Huber Loss	A combination of MSE and MAE. It is parametric loss function.	$\mathcal{L}_{\text{Huberloss}} = \begin{cases} \frac{1}{2}(y_i - f(x_i))^2 & : y_i - f(x_i) \leq \delta \\ \delta(y_i - f(x_i) - \frac{1}{2}\delta) & : \text{otherwise} \end{cases}$
Log Cosh Loss	Similar to Huber Loss + non-parametric. But computationally expensive.	$\mathcal{L}_{\text{LogCosh}} = \frac{1}{N} \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$

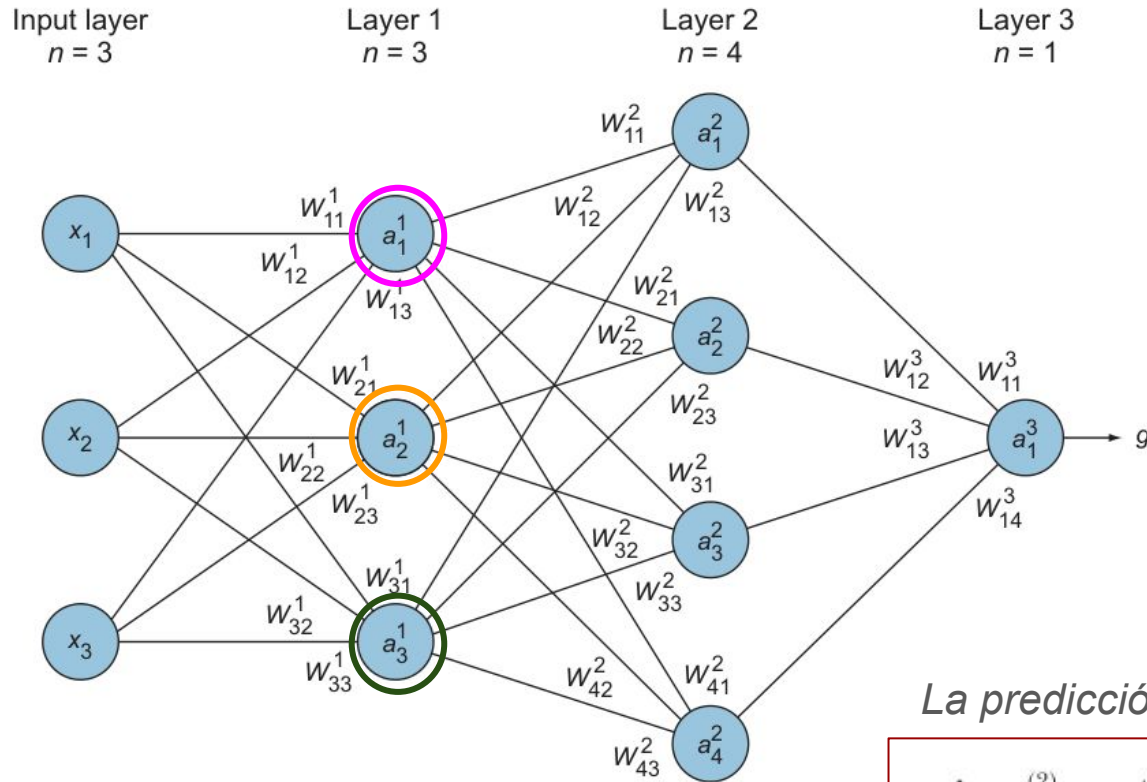
Loss functions para clasificación

Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = \frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(x_i)) + (1 - y_i) \cdot \log(1 - p(x_i))$
Hinge Loss	Penalizes wrong and right (but less confident) predictions. Commonly used in SVMs.	$\mathcal{L}_{\text{Hinge}} = \max(0, 1 - (f(x) \cdot y))$
Cross Entropy Loss	Extension of BCE loss to multi-class classification.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(f(x_{ij}))$ N : samples; M : classes
KL Divergence	Minimizes the divergence between predicted and true probability distribution	$\mathcal{L}_{KL} = \sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$

Proceso de aprendizaje en redes



Propagación hacia adelante



Los cálculos para la capa 1:

$$a_1^{(1)} = \sigma(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + w_{31}^{(1)} x_3)$$

$$a_2^{(1)} = \sigma(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{32}^{(1)} x_3)$$

$$a_3^{(1)} = \sigma(w_{13}^{(1)} x_1 + w_{23}^{(1)} x_2 + w_{33}^{(1)} x_3)$$

Para la capa 2:

$$a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \text{ and } a_4^{(2)}$$

La predicción en la capa 3:

$$\hat{y} = a_1^{(2)} = \sigma(w_{11}^{(3)} a_1^{(2)} + w_{12}^{(3)} a_2^{(2)} + w_{13}^{(3)} a_3^{(2)} + w_{14}^{(3)} a_4^{(2)})$$

Figure 2.20 A simple three-layer neural network

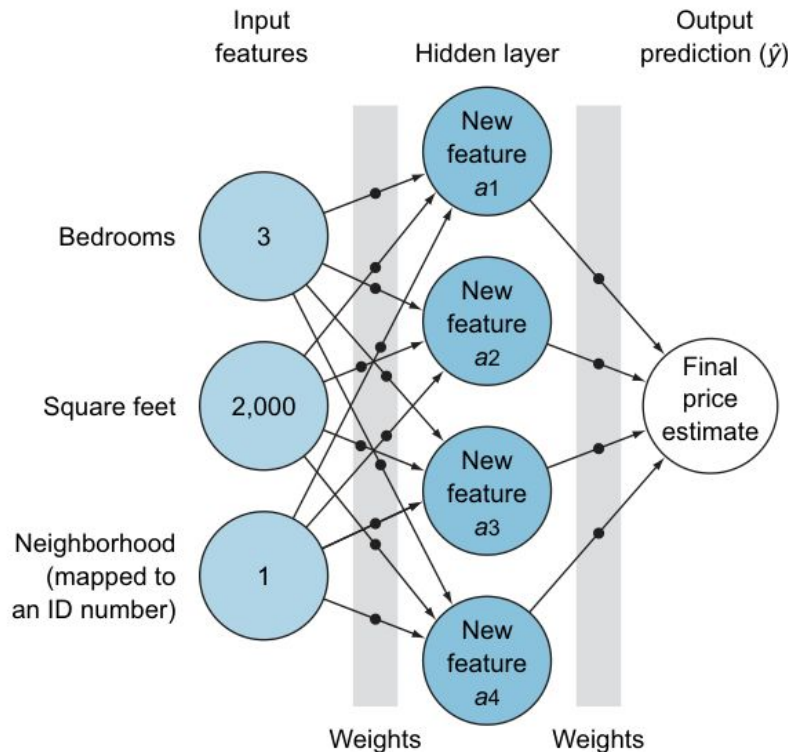
Propagación hacia adelante

$$\hat{y} = \sigma \cdot W^{(3)} \cdot \sigma \cdot W^{(2)} \cdot \sigma \cdot W^{(1)} \cdot (x)$$

$$\hat{y} = \underbrace{\sigma \begin{bmatrix} W_{11}^3 & W_{12}^3 & W_{13}^3 & W_{14}^3 \end{bmatrix}}_{\text{Layer 3}} \cdot \underbrace{\sigma \begin{bmatrix} W_{11}^2 & W_{12}^2 & W_{13}^2 \\ W_{21}^2 & W_{22}^2 & W_{23}^2 \\ W_{31}^2 & W_{32}^2 & W_{33}^2 \\ W_{41}^2 & W_{42}^2 & W_{43}^2 \end{bmatrix}}_{\text{Layer 2}} \cdot \underbrace{\sigma \begin{bmatrix} W_{11}^1 & W_{12}^1 & W_{13}^1 \\ W_{21}^1 & W_{22}^1 & W_{23}^1 \\ W_{31}^1 & W_{32}^1 & W_{33}^1 \end{bmatrix}}_{\text{Layer 1}} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{\text{Input vector}}$$

Figure 2.21 Reading from left to right, we stack the inputs together in one vector, multiply the input vector by the weights matrix from layer 1, apply the sigmoid function, and multiply the result.

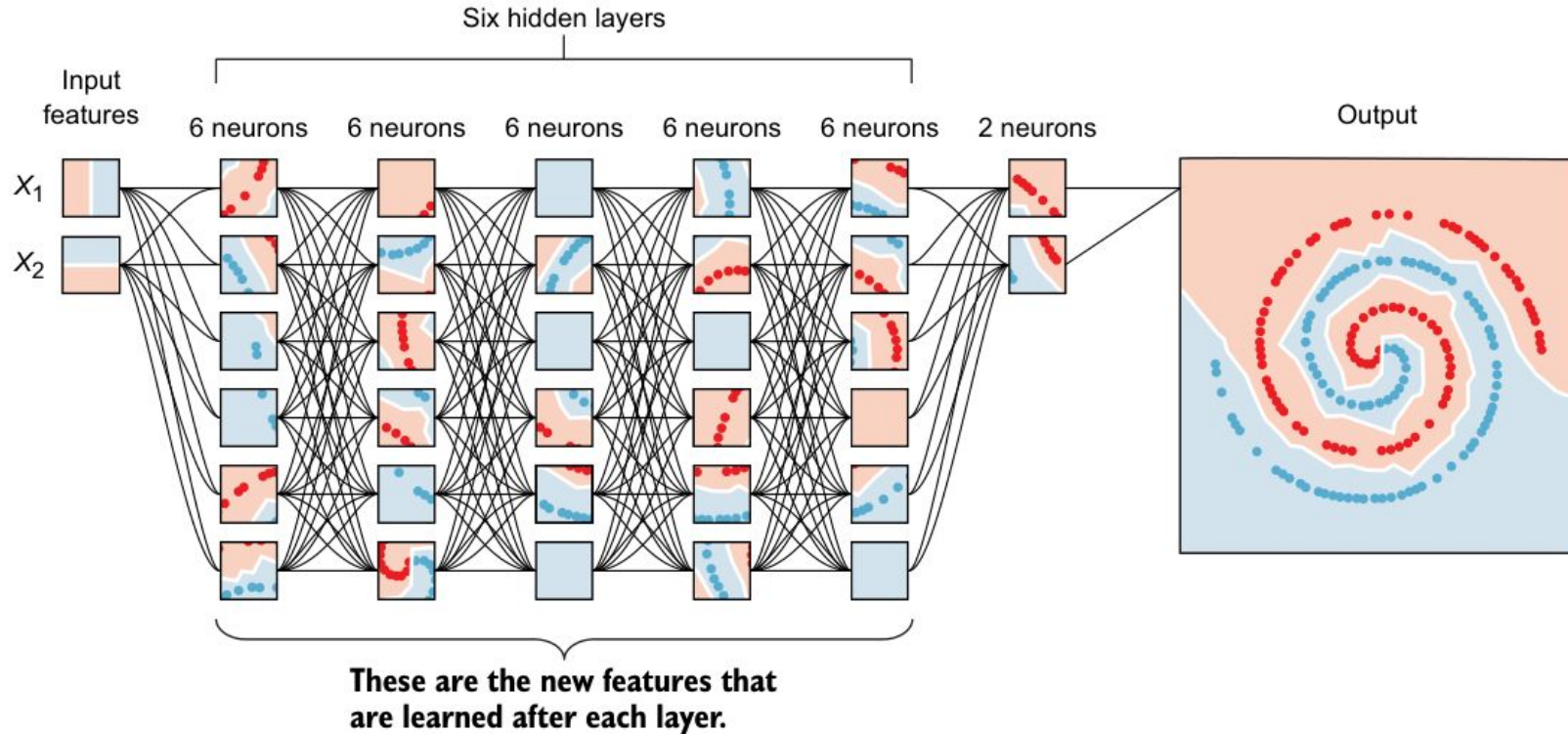
Cómo se aprende en una red feedforward



Aprendizaje de features en capas ocultas

Figure 2.22 A small neural network to estimate the price of a house based on three features: how many bedrooms it has, how big it is, and which neighborhood it is in

Cómo se aprende en una red feedforward



Backpropagation

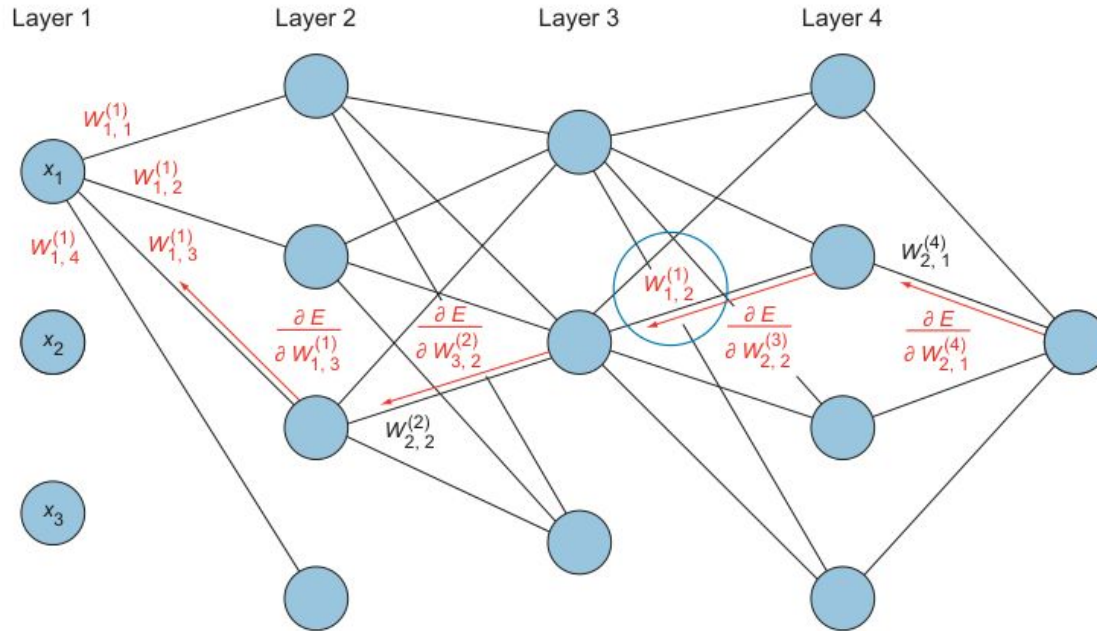
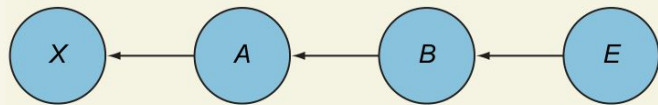


Figure 2.35 Backpropagation becomes complicated when we have a multilayer perceptron (MLP) network with many weight variables.

Regla de la cadena

$$\begin{aligned}\frac{d}{dx} f(g(x)) &= \frac{d}{dx} \text{ outside function} \times \frac{d}{dx} \text{ inside function} \\ &= \frac{d}{dx} f(g(x)) \times \frac{d}{dx} g(x)\end{aligned}$$



$$\frac{dE}{dx} = \frac{dE}{dB} \cdot \frac{dB}{dA} \cdot \frac{dA}{dx}$$

La **Regla de la Cadena** (*Chain Rule*) es el fundamento detrás del algoritmo de **Backpropagation**.

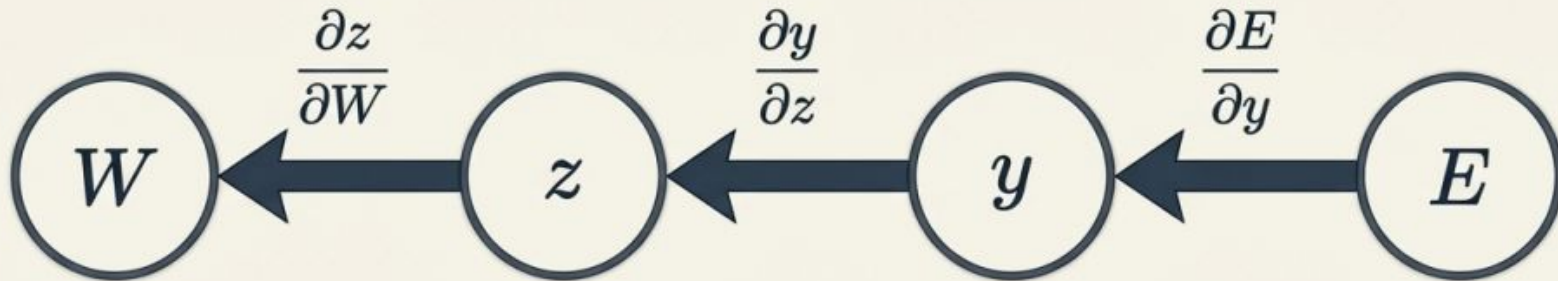
Su función principal es hacer posible el aprendizaje en la red:

Cálculo del Gradiente: Permite calcular el gradiente (la derivada del error con respecto a los pesos) de manera eficiente.

Propagación del Error: El error, determinado en la capa de salida tras el *forward pass*, debe ser distribuido a través de todas las capas ocultas hasta la capa de entrada. La Regla de la Cadena permite descomponer la derivada del error total en contribuciones parciales para cada peso individual de la red.

Actualización de Pesos: Al calcular cuánto contribuyó un peso específico al error final, el algoritmo sabe exactamente cómo ajustar ese peso (a través de un optimizador como Adam o SGD) para reducir el error en la siguiente iteración, haciendo que la red "aprenda".

Regla de la cadena



$$\frac{\partial E}{\partial W} = \left(\frac{\partial E}{\partial y} \right) \times \left(\frac{\partial y}{\partial z} \right) \times \left(\frac{\partial z}{\partial W} \right)$$

Forward y backward pass

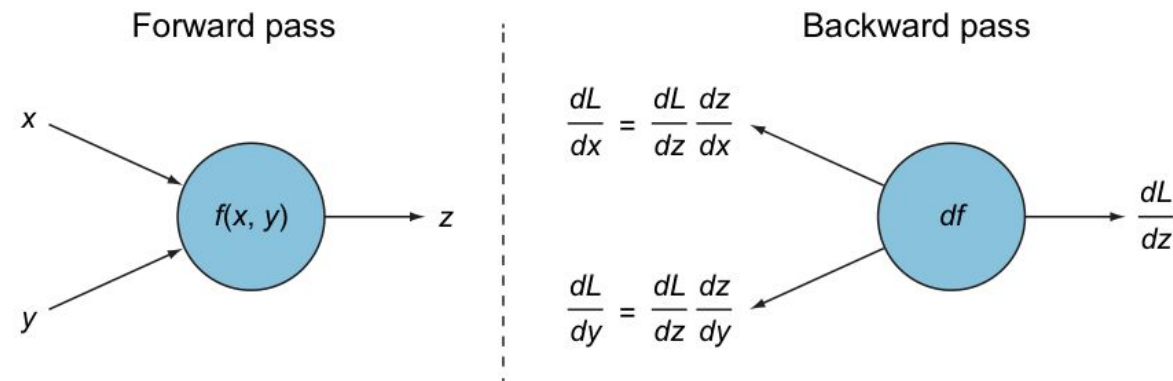


Figure 2.37 The forward pass calculates the output prediction (left). The backward pass passes the derivative of the error backward to update its weights (right).

Fase de cálculo y
predicción

Fase de corrección y
aprendizaje

El **forward pass** es la fase de **cálculo y predicción**.

Propósito: Transformar la entrada de datos en una predicción de salida.

Mecanismo: Es un flujo **unidireccional** de tensores (datos) a través de la red. La información viaja desde la capa de entrada, pasa por las capas ocultas, hasta llegar a la capa de salida.

Resultado: Genera la predicción de la red, que luego es comparada con el valor real mediante una Función de Pérdida (*Loss Function*) para determinar el error.

Forward y backward pass

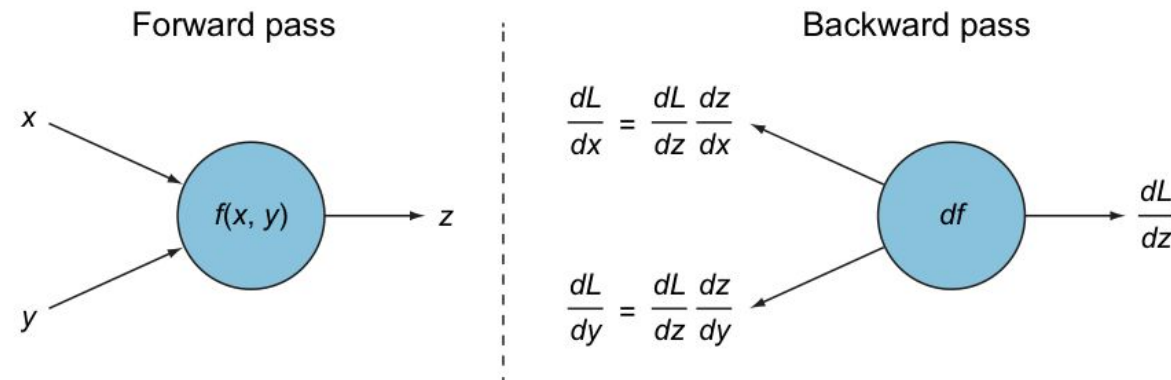


Figure 2.37 The forward pass calculates the output prediction (left). The backward pass passes the derivative of the error backward to update its weights (right).

Fase de cálculo y
predicción

Fase de corrección y
aprendizaje

El **backward pass** es la fase de **corrección y aprendizaje**.

Propósito: Calcular los gradientes del error con respecto a los pesos de la red y utilizarlos para ajustar dichos pesos.

Mecanismo: La información (el error calculado en el *forward pass*) se propaga **hacia atrás** desde la capa de salida hacia la capa de entrada. Utiliza el **algoritmo de la cadena** (basado en derivadas) para determinar cuánto contribuyó cada peso individual al error total.

Resultado: Permite la **actualización de pesos** a través de un optimizador (como SGD, Adam, etc.) para que la red cometa un error menor en la siguiente iteración, haciendo que la red "aprenda".

Forward y backward pass

Forward Pass

$$X \rightarrow Y_{pred}$$

$$z = W a + b \quad a = f(z)$$

Backward Pass

$$E \rightarrow \nabla W$$

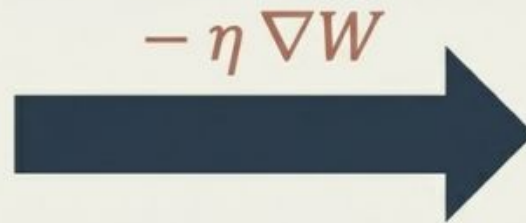
$$\frac{\partial E}{\partial W} = \frac{\partial E}{\partial y} \times \frac{\partial y}{\partial z} \times \frac{\partial z}{\partial W}$$

- El Perceptrón Multicapa no es una caja negra mágica.
- Es un sistema matemático elegante de dos vías: evalúa funciones compuestas no lineales hacia adelante y distribuye correcciones derivadas hacia atrás.
- La suma de bloques de construcción simples operando en paralelo es la verdadera escala de la inteligencia artificial moderna.

Forward y backward pass

W_{viejo}

0.5	-0.3	0.1
0.2	0.8	-0.4
-0.1	0.6	0.3

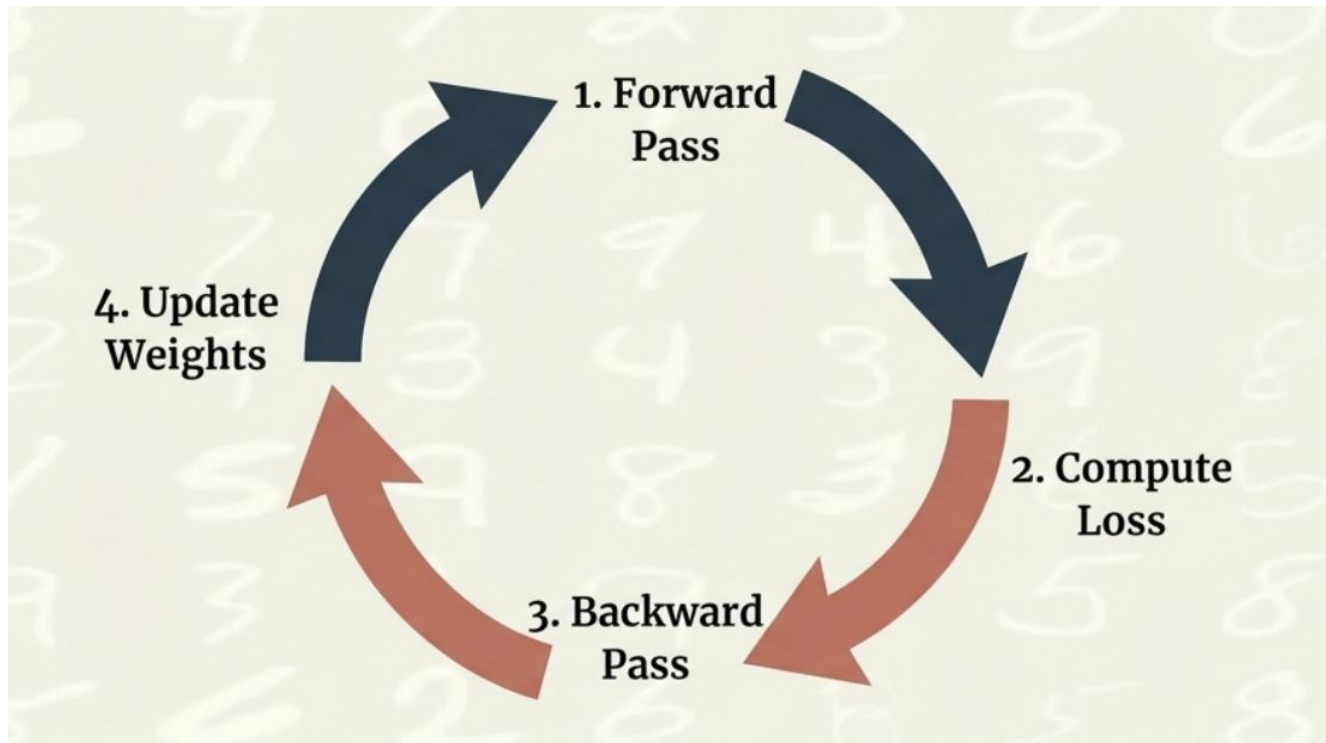


W_{nuevo}

0.48	-0.32	0.11
0.21	0.79	-0.39
-0.11	0.61	0.29

Ecuación General: $W_{nuevo} = W_{viejo} - \eta \nabla W$

Pasos del entrenamiento



Demo

Entrenamiento de redes neuronales



Epoch
000,000

Learning rate

0.03

Activation

ReLU

Regularization

L1

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1
 X_2
 X_1^2
 X_2^2
 $X_1 X_2$
 $\sin(X_1)$
 $\sin(X_2)$

2 HIDDEN LAYERS

6 neurons

5 neurons

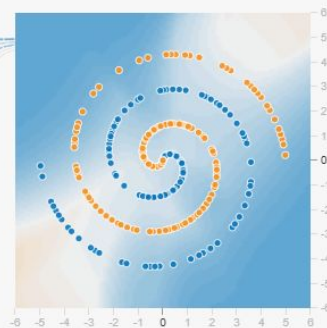
This is the output from one neuron. Hover to see it larger.

The outputs are mixed with varying weights, shown by the thickness of the lines.

OUTPUT

Test loss 0.578

Training loss 0.604



Colors shows data, neuron and weight values.



☐ Show test data ☐ Discretize output

- <https://www.datacamp.com/fr/blog/generative-vs-discriminative-models>
- <https://blog.dailydoseofds.com/p/a-visual-and-intuitive-guide-to-what>
- <https://medium.com/@derek246810/building-an-xor-neural-network-from-scratch-learn-from-the-basics-63a2a22495ae>
- <https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-deep-learning-optimizers/>
- <https://www.cse.iitm.ac.in/~miteshk/CS6910.html> (syllabus del curso)
- <https://towardsdatascience.com/hopfield-networks-neural-memory-machines-4c94be821073/>
- <https://www.datacamp.com/tutorial/introduction-to-activation-functions-in-neural-networks>
- <https://www.datacamp.com/tutorial/gelu>
- <https://ml-explained.com/blog/activation-functions-explained>
- <https://www.youtube.com/watch?v=ScGmrFBmoVI>
- <https://www.datacamp.com/tutorial/softplus>

- <https://medium.com/@manndsylva/from-theory-to-practice-comparing-the-mcculloch-pitts-and-rosenblatt-perceptron-models-d5c01da507e9>
- Mish: A Self Regularized Non-Monotonic Activation Function <https://arxiv.org/abs/1908.08681>
- Deep Sparse Rectifier Units <https://proceedings.mlr.press/v15/glorot11a/glorot11a.pdf>